

STAR RX72N - encoder_test Bench Test
1.0.0

Generated by Doxygen 1.16.1

1 Directory Hierarchy	1
1.1 Directories	1
2 File Index	3
2.1 File List	3
3 Directory Documentation	5
3.1 encoder_test Directory Reference	5
4 File Documentation	7
4.1 encoder_test/clock.c File Reference	7
4.1.1 Detailed Description	8
4.1.2 Enumeration Type Documentation	8
4.1.2.1 clock_byte_constants_t	8
4.1.2.2 clock_extra_addrs_t	9
4.1.2.3 clock_sckcr_t	9
4.1.2.4 clock_timeout_t	9
4.1.2.5 clock_word_constants_t	10
4.1.3 Function Documentation	10
4.1.3.1 clock_init()	10
4.2 clock.c	11
4.3 encoder_test/linker.ld File Reference	13
4.4 linker.ld	13
4.5 encoder_test/main.c File Reference	14
4.5.1 Detailed Description	16
4.5.2 Data Structure Documentation	16
4.5.2.1 struct motor_wiring_t	16
4.5.2.2 struct enc_pin_t	17
4.5.3 Macro Definition Documentation	17
4.5.3.1 REG16_AT	17
4.5.3.2 REG32_AT	18
4.5.3.3 REG8	18
4.5.3.4 REG8_AT	18
4.5.4 Enumeration Type Documentation	18
4.5.4.1 delay_const_t	18
4.5.4.2 enc_const_t	19
4.5.4.3 port_reg_base_t	19
4.5.4.4 status_led_pins_t	19
4.5.4.5 test_constants_t	20
4.5.4.6 test_timing_t	20
4.5.5 Function Documentation	20
4.5.5.1 busy_wait_ms()	20
4.5.5.2 clock_init()	21
4.5.5.3 encoder_read_raw()	22

4.5.5.4 encoders_init()	22
4.5.5.5 error_park()	23
4.5.5.6 gpio_set_output()	24
4.5.5.7 gpio_write()	25
4.5.5.8 led()	25
4.5.5.9 leds_init()	26
4.5.5.10 main()	27
4.5.5.11 motor_drv_gpio_init()	29
4.5.5.12 motor_pwm_init()	30
4.5.5.13 port_pdr()	31
4.5.5.14 port_pmr()	31
4.5.5.15 port_podr()	31
4.5.5.16 print_counts()	32
4.5.5.17 sci9_debug_init()	32
4.5.5.18 sci9_debug_puthex16()	33
4.5.5.19 sci9_debug_puts()	34
4.5.6 Variable Documentation	35
4.5.6.1 k_enc_pins	35
4.5.6.2 k_motors	35
4.6 main.c	36
4.7 encoder_test/sci9.c File Reference	42
4.7.1 Enumeration Type Documentation	43
4.7.1.1 hex_shifts_t	43
4.7.1.2 sci9_addrs_t	44
4.7.1.3 sci9_cfg_t	44
4.7.1.4 sci9_settle_t	45
4.7.2 Function Documentation	45
4.7.2.1 sci9_brr()	45
4.7.2.2 sci9_debug_init()	45
4.7.2.3 sci9_debug_putc()	46
4.7.2.4 sci9_debug_puthex16()	47
4.7.2.5 sci9_debug_puthex32()	48
4.7.2.6 sci9_debug_puts()	48
4.7.2.7 sci9_scmr()	49
4.7.2.8 sci9_scr()	49
4.7.2.9 sci9_semr()	49
4.7.2.10 sci9_smr()	50
4.7.2.11 sci9_ssr()	50
4.7.2.12 sci9_tdr()	51
4.8 sci9.c	51
4.9 encoder_test/stubs.c File Reference	53
4.9.1 Detailed Description	54
4.9.2 Function Documentation	54

4.9.2.1 rx_log_uart_putc()	54
4.9.2.2 rx_log_uart_puthex()	55
4.9.2.3 rx_log_uart_putint()	55
4.9.2.4 rx_log_uart_puts()	55
4.9.2.5 rx_log_uart_putuint()	55
4.9.2.6 uart_debug_putc()	55
4.9.2.7 uart_debug_puthex()	55
4.9.2.8 uart_debug_putint()	56
4.9.2.9 uart_debug_puts()	56
4.9.2.10 uart_debug_putuint()	56
4.10 stubs.c	56

Chapter 1

Directory Hierarchy

1.1 Directories

encoder_test	5
clock.c	7
linker.ld	13
main.c	14
sci9.c	42
stubs.c	53

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

encoder__test/ clock.c	
RX72N clock init for the production STAR PCB: MOSC 24 MHz crystal -> PLL 240 MHz -> ICLK=240 / PCKA=120 / PCKB=60 / FCLK=60 MHz	7
encoder__test/ linker.ld	13
encoder__test/ main.c	
Closed-loop encoder verification test	14
encoder__test/ sci9.c	42
encoder__test/ stubs.c	
No-op log/UART stubs for the motor spin test	53

Chapter 3

Directory Documentation

3.1 encoder_test Directory Reference

Files

- file [clock.c](#)
RX72N clock init for the production STAR PCB: MOSC 24 MHz crystal -> PLL 240 MHz -> ICLK=240 / PCKA=120 / PCKB=60 / FCLK=60 MHz.
- file [linker.ld](#)
- file [main.c](#)
Closed-loop encoder verification test.
- file [sci9.c](#)
- file [stubs.c](#)
No-op log/UART stubs for the motor spin test.

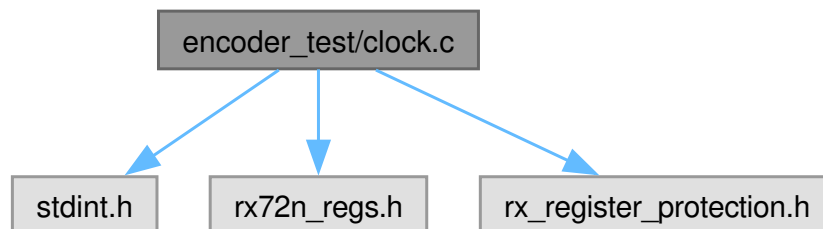
Chapter 4

File Documentation

4.1 encoder_test/clock.c File Reference

RX72N clock init for the production STAR PCB: MOSC 24 MHz crystal -> PLL 240 MHz -> ICLK=240 / PCKA=120 / PCKB=60 / FCLK=60 MHz.

```
#include <stdint.h>
#include "rx72n_regs.h"
#include "rx_register_protection.h"
Include dependency graph for clock.c:
```



Enumerations

- enum [clock_extra_addrs_t](#) : uintptr_t { [k_packcr_addr](#) = 0x00080044U }
- enum [clock_word_constants_t](#) : uint16_t { [k_pllcr_mosc_x10](#) = 0x1300U }
- enum [clock_sckcr_t](#) : uint32_t { [k_sckcr_value](#) = 0x21C21211U }
- enum [clock_byte_constants_t](#) : uint8_t {
 [k_sub_clock_stopped](#) = 0x01U , [k_main_osc_enabled](#) = 0x00U , [k_pll_enabled](#) = 0x00U ,
 [k_oscovfsr_moovf](#) = 0x01U ,
 [k_oscovfsr_plovf](#) = 0x04U }
- enum [clock_timeout_t](#) : uint32_t { [k_main_osc_poll_max](#) = 500000U , [k_pll_lock_poll_max](#) = 500000U }

Functions

- void [clock_init](#) (void)

4.1.1 Detailed Description

RX72N clock init for the production STAR PCB: MOSC 24 MHz crystal -> PLL 240 MHz -> ICLK=240 / PCKA=120 / PCKB=60 / FCLK=60 MHz.

Self-contained equivalent of `src/rx_clock_power_init.c`, stripped of the logging, asserts, simulator branches, and PLL/USB clock paths the motor spin test does not need. Configures only the main PLL.

Path: EXTAL 24 MHz -> MOSC -> PLL ($\times 10 / 1$) = 240 MHz -> SCKCR=0x21C21211. After this routine returns, ICLK is 240 MHz and PCLKA (the GPTW source) is 120 MHz, matching `k_pclka_hz` in `libs/rx_hal/inc/rx72n_clock.h`.

SPDX-License-Identifier: MIT

Copyright

Copyright (c) 2026 Locked Inc.

Definition in file [clock.c](#).

4.1.2 Enumeration Type Documentation

4.1.2.1 `clock_byte_constants_t`

enum [clock_byte_constants_t](#) : `uint8_t`

Enumerator

<code>k_sub_clock_stopped</code>	SOSCCR: stop sub-clock oscillator
<code>k_main_osc_enabled</code>	MOSCCR: 0 = MOSC running
<code>k_pll_enabled</code>	PLLCR2: 0 = PLL running
<code>k_oscovfsr_moovf</code>	OSCOVFSR bit 0: MOSC stable
<code>k_oscovfsr_plovf</code>	OSCOVFSR bit 2: PLL stable

Definition at line 58 of file [clock.c](#).

```
00058      : uint8_t {
00059  k_sub_clock_stopped = 0x01U, /**< SOSCCR: stop sub-clock oscillator */
00060  k_main_osc_enabled  = 0x00U, /**< MOSCCR: 0 = MOSC running */
00061  k_pll_enabled       = 0x00U, /**< PLLCR2: 0 = PLL running */
00062  k_oscovfsr_moovf    = 0x01U, /**< OSCOVFSR bit 0: MOSC stable */
00063  k_oscovfsr_plovf    = 0x04U, /**< OSCOVFSR bit 2: PLL stable */
00064 } clock_byte_constants_t;
```

4.1.2.2 clock_extra_addrs_t

```
enum clock_extra_addrs_t : uintptr_t
```

Enumerator

k_packcr_addr	Peripheral clock source: 0=PLL
---------------	--------------------------------

Definition at line 27 of file [clock.c](#).

```
00027     : uintptr_t {
00028   k_packcr_addr = 0x00080044U, /**< Peripheral clock source: 0=PLL */
00029 } clock_extra_addrs_t;
```

4.1.2.3 clock_sckcr_t

```
enum clock_sckcr_t : uint32_t
```

Enumerator

k_sckcr_value	
---------------	--

Definition at line 49 of file [clock.c](#).

```
00049     : uint32_t {
00050   /*
00051    * SCKCR dividers (matches production k_system_clock_dividers):
00052    *   FCK=/4 (60), ICK=/1 (240), PSTOP1=1, PSTOP0=1, BCK=/4 (60),
00053    *   PCKA=/2 (120), PCKB=/4 (60), PCKC=/4 (60), PCKD=/4 (60).
00054    */
00055   k_sckcr_value = 0x21C21211U,
00056 } clock_sckcr_t;
```

4.1.2.4 clock_timeout_t

```
enum clock_timeout_t : uint32_t
```

Enumerator

k_main_osc_poll_max	
k_pll_lock_poll_max	

Definition at line 66 of file [clock.c](#).

```
00066     : uint32_t {
00067   /* Bounded poll counts (NASA Rule 2). We poll the hardware ready flag
00068    * (OSCOVFSR.MOOVF / .PLOVF) so the actual wait is hardware-determined --
00069    * roughly 10 ms physical time regardless of CPU clock. The upper bound
00070    * just prevents an infinite spin if the crystal is missing. ~500k iters
00071    * at LOCO 240 kHz is ~3 s, which is still much shorter than the prior
00072    * fixed 10-second busy-wait. */
00073   k_main_osc_poll_max = 500000U,
00074   k_pll_lock_poll_max = 500000U,
00075 } clock_timeout_t;
```

4.1.2.5 clock_word_constants_t

enum `clock_word_constants_t` : `uint16_t`

Enumerator

k_pllcr_mosc_x10	
------------------	--

Definition at line 34 of file `clock.c`.

```

00034         : uint16_t {
00035     /*
00036     * PLLCR layout:
00037     * bits[1:0] PLIDIV : 00 = /1
00038     * bit [4] PLLSRCSEL : 0 = MOSC, 1 = HOCO
00039     * bits[13:8] STC : multiplier = (STC + 1) / 2
00040     *
00041     * MOSC 24 MHz / 1 * 10 = 240 MHz. STC = (10 * 2) - 1 = 19 = 0x13.
00042     * PLLSRCSEL = 0 (MOSC) -> bit 4 clear.
00043     *
00044     * Production rx_clock_power_init.c uses k_pll_multiplier_10_div_1 = 0x1300.
00045     */
00046     k_pllcr_mosc_x10 = 0x1300U,
00047 } clock_word_constants_t;

```

4.1.3 Function Documentation

4.1.3.1 clock_init()

void `clock_init` (
 void)

Definition at line 80 of file `clock.c`.

```

00081 {
00082     volatile rx_system_regs_t *sys = system_regs();
00083
00084     /* Unlock PRC0 (clock) + PRC1 (module stop) write protection. */
00085     *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00086
00087     /* Stop sub-clock oscillator -- not used on STAR. */
00088     sys->sosccr = k_sub_clock_stopped;
00089
00090     /* Start MOSC (24 MHz external crystal). */
00091     sys->mosccr = k_main_osc_enabled;
00092
00093     /* Poll the hardware MOSC-stable flag. Exits as soon as the crystal has
00094     * settled (typically ~10 ms physical time) instead of waiting a fixed
00095     * 2.4M cycles, which on the LOCO 240 kHz boot clock is ~10 seconds. */
00096     for (volatile uint32_t i = 0; i < k_main_osc_poll_max; i++) {
00097         if ((sys->oscofsr & k_oscofsr_moovf) != 0U) {
00098             break;
00099         }
00100     }
00101
00102     /* Configure and enable PLL: 24 MHz x 10 / 1 = 240 MHz. */
00103     sys->pllcr = k_pllcr_mosc_x10;
00104     sys->pllcr2 = k_pll_enabled;
00105
00106     /* Wait for PLL lock (bounded). If we time out, the crystal is missing
00107     * or not driving -- DO NOT switch SKCR3 to PLL or the chip will halt
00108     * on a dead clock. Stay on LOCO so the LEDs still respond and the user
00109     * can see we got past clock_init. */
00110     bool pll_locked = false;
00111     for (volatile uint32_t i = 0; i < k_pll_lock_poll_max; i++) {
00112         if ((sys->oscofsr & k_oscofsr_plovf) != 0U) {
00113             pll_locked = true;
00114             break;
00115         }
00116     }
00117
00118     if (pll_locked) {
00119         /* MANDATORY: raise flash wait state BEFORE switching to >120 MHz. */
00120         *memwait_reg() = k_memwait_one_wait;
00121     }

```

```

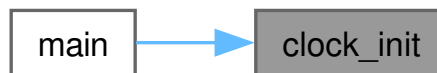
00122  /* Apply system clock dividers. */
00123  sys->sckcr = k_sckcr_value;
00124
00125  /* Route UCLK from main PLL (UPLLSEL=0). */
00126  *((volatile uint16_t *)k_packcr_addr) = 0x0000U;
00127
00128  /* Switch system clock source to PLL. */
00129  sys->sckcr3 = k_sckcr3_ksel_pll;
00130  }
00131
00132  /* Re-lock PRCR. */
00133  *prcr_reg() = k_rx_prcr_lock;
00134  }

```

References [k_main_osc_enabled](#), [k_main_osc_poll_max](#), [k_oscovfsr_moovf](#), [k_oscovfsr_plovf](#), [k_packcr_addr](#), [k_pll_enabled](#), [k_pll_lock_poll_max](#), [k_pllcr_mosc_x10](#), [k_sckcr_value](#), and [k_sub_clock_stopped](#).

Referenced by [main\(\)](#).

Here is the caller graph for this function:



4.2 clock.c

[Go to the documentation of this file.](#)

```

00001 /**
00002  * @file clock.c
00003  * @brief RX72N clock init for the production STAR PCB: MOSC 24 MHz crystal
00004  *        -> PLL 240 MHz -> ICLK=240 / PCKA=120 / PCKB=60 / FCLK=60 MHz.
00005  *
00006  * @details
00007  * Self-contained equivalent of src/rx_clock_power_init.c, stripped of the
00008  * logging, asserts, simulator branches, and PPLL/USB clock paths the motor
00009  * spin test does not need. Configures only the main PLL.
00010  *
00011  * Path: EXTAL 24 MHz -> MOSC -> PLL (x10 / 1) = 240 MHz -> SCKCR=0x21C21211.
00012  * After this routine returns, ICLK is 240 MHz and PCLKA (the GPTW source)
00013  * is 120 MHz, matching k_pclka_hz in libs/rx_hal/inc/rx72n_clock.h.
00014  *
00015  * SPDX-License-Identifier: MIT
00016  * @copyright Copyright (c) 2026 Locked Inc.
00017  */

```

```

00018
00019 #include <stdint.h>
00020

```

```

00021 #include "rx72n_regs.h"
00022 #include "rx_register_protection.h"
00023

```

```

00024 /*

```

```

=====
00025 * Register addresses not exposed by rx_system_regs_t

```

```

00026 *
=====
*/

```

```

00027 typedef enum : uintptr_t {
00028  k_packcr_addr = 0x00080044U, /**< Peripheral clock source: 0=PLL */
00029 } clock_extra_addrs_t;
00030
00031 /*

```

```

=====
00032 * Bit / value constants (mirrors src/rx_clock_power_init.c)

```

```

00033 *
=====
*/

```

```

00034 typedef enum : uint16_t {
00035  /*
00036  * PLLCR layout:
00037  * bits[1:0] PLIDIV : 00 = /1
00038  * bit [4] PLLSRCSEL : 0 = MOSC, 1 = HOCO
00039  * bits[13:8] STC : multiplier = (STC + 1) / 2

```

```

00040  *
00041  * MOSC 24 MHz / 1 * 10 = 240 MHz. STC = (10 * 2) - 1 = 19 = 0x13.
00042  * PLLSRCSEL = 0 (MOSC) -> bit 4 clear.
00043  *
00044  * Production rx_clock_power_init.c uses k_pll_multiplier_10_div_1 = 0x1300.
00045  */
00046  k_pllcr_mosc_x10 = 0x1300U,
00047 } clock_word_constants_t;
00048
00049 typedef enum : uint32_t {
00050  /*
00051  * SCKCR dividers (matches production k_system_clock_dividers):
00052  *   FCK=/4 (60), ICK=/1 (240), PSTOP1=1, PSTOP0=1, BCK=/4 (60),
00053  *   PCKA=/2 (120), PCKB=/4 (60), PCKC=/4 (60), PCKD=/4 (60).
00054  */
00055  k_sckcr_value = 0x21C21211U,
00056 } clock_sckcr_t;
00057
00058 typedef enum : uint8_t {
00059  k_sub_clock_stopped = 0x01U, /**< SOSCCR: stop sub-clock oscillator */
00060  k_main_osc_enabled = 0x00U, /**< MOSCCR: 0 = MOSC running */
00061  k_pll_enabled = 0x00U, /**< PLLCR2: 0 = PLL running */
00062  k_oscovfsr_moovf = 0x01U, /**< OSCOVFSR bit 0: MOSC stable */
00063  k_oscovfsr_plovf = 0x04U, /**< OSCOVFSR bit 2: PLL stable */
00064 } clock_byte_constants_t;
00065
00066 typedef enum : uint32_t {
00067  /* Bounded poll counts (NASA Rule 2). We poll the hardware ready flag
00068  * (OSCOVFSR.MOOVF / .PLOVF) so the actual wait is hardware-determined --
00069  * roughly 10 ms physical time regardless of CPU clock. The upper bound
00070  * just prevents an infinite spin if the crystal is missing. ~500k iters
00071  * at LOCO 240 kHz is ~3 s, which is still much shorter than the prior
00072  * fixed 10-second busy-wait. */
00073  k_main_osc_poll_max = 500000U,
00074  k_pll_lock_poll_max = 500000U,
00075 } clock_timeout_t;
00076
00077 /*
=====
00078  * clock_init -- bring the chip from POR defaults to PLL 240 / PCKA 120 MHz.
00079  *
=====
*/
00080 void clock_init(void)
00081 {
00082  volatile rx_system_regs_t *sys = system_regs();
00083
00084  /* Unlock PRC0 (clock) + PRC1 (module stop) write protection. */
00085  *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00086
00087  /* Stop sub-clock oscillator -- not used on STAR. */
00088  sys->sosccr = k_sub_clock_stopped;
00089
00090  /* Start MOSC (24 MHz external crystal). */
00091  sys->mosccr = k_main_osc_enabled;
00092
00093  /* Poll the hardware MOSC-stable flag. Exits as soon as the crystal has
00094  * settled (typically ~10 ms physical time) instead of waiting a fixed
00095  * 2.4M cycles, which on the LOCO 240 kHz boot clock is ~10 seconds. */
00096  for (volatile uint32_t i = 0; i < k_main_osc_poll_max; i++) {
00097    if ((sys->oscovfsr & k_oscovfsr_moovf) != 0U) {
00098      break;
00099    }
00100  }
00101
00102  /* Configure and enable PLL: 24 MHz x 10 / 1 = 240 MHz. */
00103  sys->pllcr = k_pllcr_mosc_x10;
00104  sys->pllcr2 = k_pll_enabled;
00105
00106  /* Wait for PLL lock (bounded). If we time out, the crystal is missing
00107  * or not driving -- DO NOT switch SCKCR3 to PLL or the chip will halt
00108  * on a dead clock. Stay on LOCO so the LEDs still respond and the user
00109  * can see we got past clock_init. */
00110  bool pll_locked = false;
00111  for (volatile uint32_t i = 0; i < k_pll_lock_poll_max; i++) {
00112    if ((sys->oscovfsr & k_oscovfsr_plovf) != 0U) {
00113      pll_locked = true;
00114      break;
00115    }
00116  }
00117
00118  if (pll_locked) {
00119    /* MANDATORY: raise flash wait state BEFORE switching to >120 MHz. */
00120    *memwait_reg() = k_memwait_one_wait;
00121
00122    /* Apply system clock dividers. */
00123    sys->sckcr = k_sckcr_value;

```

```

00124
00125 /* Route UCLK from main PLL (UPLLSEL=0). */
00126 *((volatile uint16_t *)k_packcr_addr) = 0x0000U;
00127
00128 /* Switch system clock source to PLL. */
00129 sys->sckcr3 = k_sckcr3_cksel_pll;
00130 }
00131
00132 /* Re-lock PRCR. */
00133 *prcr_reg() = k_rx_prcr_lock;
00134 }

```

4.3 encoder_test/linker.ld File Reference

4.4 linker.ld

[Go to the documentation of this file.](#)

```

00001 /*
00002  * gpio_test/linker.ld -- Linker script for GPIO breakout board test.
00003  *
00004  * Same memory map as usb_test: 512 KB internal SRAM, 2 MB internal ROM.
00005  * Includes Renesas-syntax sections (P, B, D, C) needed by ThreadX RXv3 port.
00006  */
00007
00008 MEMORY
00009 {
00010     RAM : ORIGIN = 0x00000004, LENGTH = 0x7FFFC /* 512 KB - 4 bytes */
00011     ROM : ORIGIN = 0xFFE00000, LENGTH = 0x200000 /* 2 MB */
00012 }
00013
00014 SECTIONS
00015 {
00016     /* Code -- explicit ROM base so section VMA is anchored correctly. */
00017     .text 0xFFE00000 : AT(0xFFE00000)
00018     {
00019         *(.text*)
00020         *(P)
00021         *(P_1)
00022         *(.rodata*)
00023         *(C)
00024         *(C_1)
00025         *(C_2)
00026         etext = .;
00027     } > ROM
00028
00029     /*
00030     * Relocatable vector table -- startup.S loads INTB with this address.
00031     * Must be 4-byte aligned; only vectors 27 (SWINT) and 28 (CMT0) are
00032     * used, but allocate 256 entries (1024 bytes) for safety.
00033     */
00034     .rvecors ALIGN(4) :
00035     {
00036         __rvecors_start = .;
00037         KEEP(*(.rvecors))
00038         __rvecors_end = .;
00039     } > ROM
00040
00041     /*
00042     * Initialised data -- LMA stays in ROM, VMA in RAM.
00043     * startup.S copies __data..edata from __mdata on boot.
00044     */
00045     __mdata = .;
00046     .data : AT(__mdata)
00047     {
00048         __data = .;
00049         *(.data*)
00050         *(D)
00051         *(D_1)
00052         *(D_2)
00053         __edata = .;
00054     } > RAM
00055
00056     /* BSS -- zeroed by startup.S */
00057     .bss :
00058     {
00059         __bss = .;
00060         *(.bss*)
00061         *(COMMON)

```

```

00062     *(B)
00063     *(B_1)
00064     *(B_2)
00065     _ebss = .;
00066     . = ALIGN(4);
00067     _end = .;
00068 } > RAM AT>RAM
00069
00070 /*
00071  * Stacks -- USP grows down from top of RAM, ISP 4 KB below that.
00072  */
00073 _ustack = ORIGIN(RAM) + LENGTH(RAM);
00074 _istack = _ustack - 0x1000;
00075
00076 /*
00077  * Exception vector table at fixed hardware address 0xFFFFF80.
00078  * 32 entries -- all unused.
00079  */
00080 .exvectors 0xFFFFF80 : AT(0xFFFFF80)
00081 {
00082     LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00083     LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00084     LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00085     LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00086     LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00087     LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00088     LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00089     LONG(0xFFFFFFFF); LONG(0xFFFFFFFF); LONG(0xFFFFFFFF);
00090 } > ROM
00091
00092 /* Fixed reset vector at 0xFFFFF8C */
00093 .fvectors 0xFFFFF8C : AT(0xFFFFF8C)
00094 {
00095     KEEP(*(fvectors))
00096 } > ROM
00097 }

```

4.5 encoder_test/main.c File Reference

Closed-loop encoder verification test.

```

#include <stdint.h>
#include <stdbool.h>
#include <stddef.h>
#include "rx_motor.h"
#include "rx_gptw.h"
#include "rx_mpc.h"
#include "rx_port_constants.h"
#include "rx_mtu_encoder.h"
#include "rx_encoder_tpu.h"
#include "rx72n_mtu_regs.h"
#include "rx72n_tpu_regs.h"
#include "rx72n_system_regs.h"
#include "rx_register_protection.h"

```

Include dependency graph for main.c:



Data Structures

- struct [motor_wiring_t](#)
- struct [enc_pin_t](#)

Macros

- #define REG8(a)
- #define REG8_AT(a)
- #define REG16_AT(a)
- #define REG32_AT(a)

Enumerations

- enum port_reg_base_t : uintptr_t { k_port_pdr_base = 0x0008C000U , k_port_podr_base = 0x0008C020U , k_port_pmr_base = 0x0008C060U }
- enum status_led_pins_t : uint8_t {
k_port_a = 10 , k_port_b = 11 , k_port_7 = 7 , k_bit_led0 = 7 ,
k_bit_led1 = 0 , k_bit_led2 = 1 , k_bit_led3 = 2 , k_bit_led4 = 1 ,
k_bit_led5 = 2 }
- enum test_constants_t : uint8_t { k_motor_count = 4 , k_port_6 = 6 , k_port_e = 14 }
- enum delay_const_t : uint32_t { k_iters_per_ms = 40000U }
- enum enc_const_t : uint16_t { k_counts_per_rev = 1364 }
- enum test_timing_t : uint32_t { k_spin_ms = 3000U , k_settle_ms = 500U , k_min_delta = 50U }

Functions

- void clock_init (void)
- void sci9_debug_init (void)
- void sci9_debug_puts (const char *s)
- void sci9_debug_puthex16 (uint16_t v)
- static volatile uint8_t * port_pdr (uint8_t port)
- static volatile uint8_t * port_podr (uint8_t port)
- static volatile uint8_t * port_pmr (uint8_t port)
- static void gpio_set_output (uint8_t port, uint8_t pin)
- static void gpio_write (uint8_t port, uint8_t pin, bool high)
- static void led (uint8_t port, uint8_t bit, bool on)
- static void leds_init (void)
- static void busy_wait_ms (uint32_t ms)
- static void motor_drv_gpio_init (void)
- static bool motor_pwm_init (rx_motor_handle_t handles[k_motor_count])
- static bool encoders_init (void)
- static bool encoder_read_raw (uint8_t motor_idx, uint16_t *out_count)
- static void print_counts (const char *label, uint16_t c[k_motor_count])
- static void error_park (const char *msg)
- int main (void)

Variables

- static const motor_wiring_t k_motors [k_motor_count]
- static const enc_pin_t k_enc_pins []

4.5.1 Detailed Description

Closed-loop encoder verification test.

Drives all 4 motors at +50 percent duty for ~3 seconds while sampling each encoder's hardware count. Stops motors, then dumps the count deltas via SCI9 UART so the host (cat /dev/ttyACM0 @ 115200) can capture and verify each encoder produced ticks.

Expected output on /dev/ttyACM0: ENCODER_TEST start init clock ok init leds ok init uart ok init drv gpio ok init pwm ok init enc ok before m0=0xFFFF m1=0xFFFF m2=0xFFFF m3=0xFFFF spinning motors at +50pct for 3 seconds after m0=0xFFFF m1=0xFFFF m2=0xFFFF m3=0xFFFF motors stopped delta m0=0xFFFF m1=0xFFFF m2=0xFFFF m3=0xFFFF summary FAIL m0,m3 (or PASS)

SPDX-License-Identifier: MIT

Copyright

Copyright (c) 2026 Locked Inc.

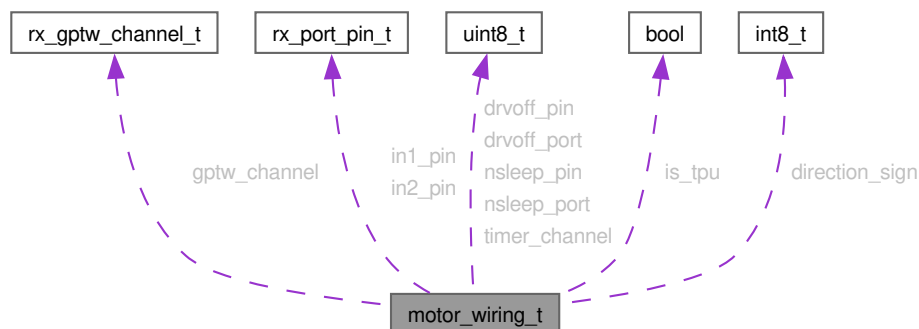
Definition in file [main.c](#).

4.5.2 Data Structure Documentation

4.5.2.1 struct motor_wiring_t

Definition at line 121 of file [main.c](#).

Collaboration diagram for motor_wiring_t:



Data Fields

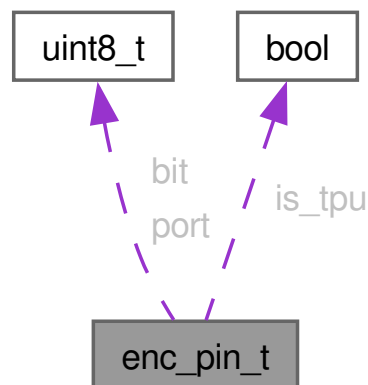
int8_t	direction_sign	
uint8_t	drvoff_pin	
uint8_t	drvoff_port	
rx_gptw_channel_t	gptw_channel	
rx_port_pin_t	in1_pin	
rx_port_pin_t	in2_pin	

bool	is_tpu	
uint8_t	nsleep_pin	
uint8_t	nsleep_port	
uint8_t	timer_channel	

4.5.2.2 struct enc_pin_t

Definition at line 237 of file [main.c](#).

Collaboration diagram for enc_pin_t:



Data Fields

uint8_t	bit	
bool	is_tpu	
uint8_t	port	

4.5.3 Macro Definition Documentation

4.5.3.1 REG16_AT

```
#define REG16_AT(  
    a)
```

Value:

```
*(volatile uint16_t *) (uintptr_t)(a))
```

Definition at line 260 of file [main.c](#).

Referenced by [encoder_read_raw\(\)](#), and [main\(\)](#).

4.5.3.2 REG32_AT

```
#define REG32_AT(  
    a)
```

Value:

```
*(volatile uint32_t *) (uintptr_t)(a))
```

Definition at line 261 of file [main.c](#).

4.5.3.3 REG8

```
#define REG8(  
    a)
```

Value:

```
*(volatile uint8_t *) (uintptr_t)(a))
```

Definition at line 53 of file [main.c](#).

Referenced by [port_pdr\(\)](#), [port_pmr\(\)](#), and [port_podr\(\)](#).

4.5.3.4 REG8_AT

```
#define REG8_AT(  
    a)
```

Value:

```
*(volatile uint8_t *) (uintptr_t)(a))
```

Definition at line 259 of file [main.c](#).

Referenced by [encoders_init\(\)](#), and [main\(\)](#).

4.5.4 Enumeration Type Documentation

4.5.4.1 delay_const_t

```
enum delay\_const\_t : uint32_t
```

Enumerator

k_iters_per_ms	
--------------------------------	--

Definition at line 165 of file [main.c](#).

```
00165     : uint32_t {  
00166     /* Empirically calibrated: 40000 iters/ms at 240 MHz with -O1 volatile loop  
00167      * (~6 cycles per iteration). 240000 was 6x too slow. */  
00168     k_iters_per_ms = 40000U,  
00169 } delay_const_t;
```

4.5.4.2 enc_const_t

enum [enc_const_t](#) : uint16_t

Enumerator

k_counts_per_rev	
------------------	--

Definition at line 225 of file [main.c](#).

```
00225      : uint16_t {
00226  k_counts_per_rev = 1364, /* 341 PPR x 4 (matches production) */
00227 } enc_const_t;
```

4.5.4.3 port_reg_base_t

enum [port_reg_base_t](#) : uintptr_t

Enumerator

k_port_pdr_base	
k_port_podr_base	
k_port_pmr_base	

Definition at line 55 of file [main.c](#).

```
00055      : uintptr_t {
00056  k_port_pdr_base = 0x0008C000U,
00057  k_port_podr_base = 0x0008C020U,
00058  k_port_pmr_base = 0x0008C060U,
00059 } port_reg_base_t;
```

4.5.4.4 status_led_pins_t

enum [status_led_pins_t](#) : uint8_t

Enumerator

k_port_a	
k_port_b	
k_port_7	
k_bit_led0	
k_bit_led1	
k_bit_led2	
k_bit_led3	
k_bit_led4	
k_bit_led5	

Definition at line 68 of file [main.c](#).

```
00068      : uint8_t {
00069  k_port_a = 10,
00070  k_port_b = 11,
00071  k_port_7 = 7,
00072  k_bit_led0 = 7, /* PA7 D9 -- heartbeat */
00073  k_bit_led1 = 0, /* PB0 D10 -- after init */
00074  k_bit_led2 = 1, /* P71 D11 -- motor PWM running */
00075  k_bit_led3 = 2, /* P72 D12 -- encoders all ticked */
00076  k_bit_led4 = 1, /* PB1 D13 -- test PASS */
00077  k_bit_led5 = 2, /* PB2 D14 -- test FAIL or init error */
00078 } status_led_pins_t;
```

4.5.4.5 test_constants_t

enum [test_constants_t](#) : uint8_t

Enumerator

k_motor_count	
k_port_6	
k_port_e	

Definition at line 135 of file [main.c](#).

```
00135      : uint8_t {
00136  k_motor_count = 4,
00137  k_port_6      = 6,
00138  k_port_e      = 14,
00139 } test_constants_t;
```

4.5.4.6 test_timing_t

enum [test_timing_t](#) : uint32_t

Enumerator

k_spin_ms	
k_settle_ms	
k_min_delta	

Definition at line 392 of file [main.c](#).

```
00392      : uint32_t {
00393  k_spin_ms    = 3000U, /* drive motors this long */
00394  k_settle_ms  = 500U,  /* let everything stop */
00395  k_min_delta  = 50U,   /* minimum ticks expected per encoder over k_spin_ms */
00396 } test_timing_t;
```

4.5.5 Function Documentation

4.5.5.1 busy_wait_ms()

```
void busy_wait_ms (
    uint32_t ms) [static]
```

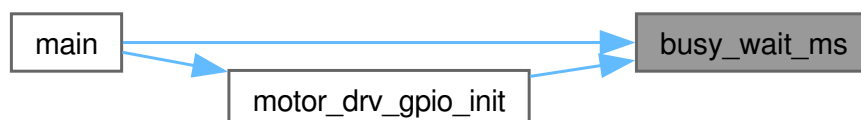
Definition at line 171 of file [main.c](#).

```
00172 {
00173  for (uint32_t m = 0; m < ms; m++) {
00174      for (volatile uint32_t i = 0; i < k_iters_per_ms; i++) {
00175          __asm__ volatile("nop");
00176      }
00177  }
00178 }
```

References [k_iters_per_ms](#).

Referenced by [main\(\)](#), and [motor_drv_gpio_init\(\)](#).

Here is the caller graph for this function:



4.5.5.2 clock_init()

```
void clock_init (
    void ) [extern]
```

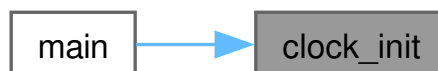
Definition at line 80 of file [clock.c](#).

```
00081 {
00082     volatile rx_system_regs_t *sys = system_regs();
00083
00084     /* Unlock PRC0 (clock) + PRC1 (module stop) write protection. */
00085     *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00086
00087     /* Stop sub-clock oscillator -- not used on STAR. */
00088     sys->sosccr = k_sub_clock_stopped;
00089
00090     /* Start MOSC (24 MHz external crystal). */
00091     sys->mosccr = k_main_osc_enabled;
00092
00093     /* Poll the hardware MOSC-stable flag. Exits as soon as the crystal has
00094      * settled (typically ~10 ms physical time) instead of waiting a fixed
00095      * 2.4M cycles, which on the LOCO 240 kHz boot clock is ~10 seconds. */
00096     for (volatile uint32_t i = 0; i < k_main_osc_poll_max; i++) {
00097         if ((sys->oscovfsr & k_oscovfsr_moovf) != 0U) {
00098             break;
00099         }
00100     }
00101
00102     /* Configure and enable PLL: 24 MHz x 10 / 1 = 240 MHz. */
00103     sys->pllcr = k_pllcr_mosc_x10;
00104     sys->pllcr2 = k_pll_enabled;
00105
00106     /* Wait for PLL lock (bounded). If we time out, the crystal is missing
00107      * or not driving -- DO NOT switch SCKCR3 to PLL or the chip will halt
00108      * on a dead clock. Stay on LOCO so the LEDs still respond and the user
00109      * can see we got past clock_init. */
00110     bool pll_locked = false;
00111     for (volatile uint32_t i = 0; i < k_pll_lock_poll_max; i++) {
00112         if ((sys->oscovfsr & k_oscovfsr_plovf) != 0U) {
00113             pll_locked = true;
00114             break;
00115         }
00116     }
00117
00118     if (pll_locked) {
00119         /* MANDATORY: raise flash wait state BEFORE switching to >120 MHz. */
00120         *memwait_reg() = k_memwait_one_wait;
00121
00122         /* Apply system clock dividers. */
00123         sys->sckcr = k_sckcr_value;
00124
00125         /* Route UCLK from main PLL (UPLLSEL=0). */
00126         *((volatile uint16_t *)k_packcr_addr) = 0x0000U;
00127
00128         /* Switch system clock source to PLL. */
00129         sys->sckcr3 = k_sckcr3_cksel_pll;
00130     }
00131
00132     /* Re-lock PRCR. */
00133     *prcr_reg() = k_rx_prcr_lock;
00134 }
```

References [k_main_osc_enabled](#), [k_main_osc_poll_max](#), [k_oscovfsr_moovf](#), [k_oscovfsr_plovf](#), [k_packcr_addr](#), [k_pll_enabled](#), [k_pll_lock_poll_max](#), [k_pllcr_mosc_x10](#), [k_sckcr_value](#), and [k_sub_clock_stopped](#).

Referenced by [main\(\)](#).

Here is the caller graph for this function:



4.5.5.3 encoder_read_raw()

```
bool encoder_read_raw (
    uint8_t motor_idx,
    uint16_t * out_count) [static]
```

Definition at line 349 of file [main.c](#).

```
00350 {
00351     /* Direct register read -- bypass the lib (rx_encoder_read_raw works
00352      * but for symmetry with the bypass init). */
00353     static const uintptr_t k_tcmt_addr[k_motor_count] = {
00354         0x000C1386U, /* MTU1 */
00355         0x000C1406U, /* MTU2 */
00356         0x00088120U + 6U, /* TPU1 */
00357         0x00088130U + 6U, /* TPU2 */
00358     };
00359     *out_count = REG16_AT(k_tcmt_addr[motor_idx]);
00360     return true;
00361 }
```

References [k_motor_count](#), and [REG16_AT](#).

Referenced by [main\(\)](#).

Here is the caller graph for this function:



4.5.5.4 encoders_init()

```
bool encoders_init (
    void ) [static]
```

Definition at line 268 of file [main.c](#).

```
00269 {
00270     /* Step 1: enable MTU (MSTPA9) + TPU (MSTPA13) module clocks. */
00271     *prcr_reg() = k_rx_prcr_unlock_all;
00272     system_regs()->mstpcra &= ~(uint32_t)((1UL << 9) | (uint32_t)k_tpu_mstpcra_mstp13);
00273     *prcr_reg() = k_rx_prcr_lock;
00274
00275     /* Step 2 + 3: RX72N MPC pin setup requires the sequence PMR=0, write PFS,
00276      * PMR=1. MTCLKA/MTCLKB on P24/P25 take PSEL=0x02 (the production header's
00277      * old k_psel_mtu_phase=0x03 was WRONG -- 0x02 is the MTCLK PSEL on RX72N).
00278      * For TPU TCLK keep our verified per-pin values. */
00279     volatile uint8_t *pwpr = (volatile uint8_t *)0x0008C11FU;
00280     volatile uint8_t *p2pmr = (volatile uint8_t *)0x0008C062U;
00281     volatile uint8_t *papmr = (volatile uint8_t *)0x0008C06AU;
00282     volatile uint8_t *pbpmr = (volatile uint8_t *)0x0008C06BU;
00283     volatile uint8_t *pcpmr = (volatile uint8_t *)0x0008C06CU;
00284
00285     /* clear PMR for all encoder input pins so PFS writes take effect */
00286     *p2pmr &= (uint8_t)~(uint8_t)((1U << 4) | (1U << 5));
00287     *papmr &= (uint8_t)~(uint8_t)((1U << 1) | (1U << 3));
00288     *pcpmr &= (uint8_t)~(uint8_t)((1U << 0) | (1U << 2) | (1U << 5));
00289     *pbpmr &= (uint8_t)~(uint8_t)(1U << 3);
00290
00291     /* PFS writes */
00292     *pwpr = 0x00U; *pwpr = 0x40U;
00293     REG8_AT(0x0008C140U + 2U * 8U + 4U) = 0x02U; /* P24 MTCLKA -- enc0 A (MTU PSEL=0x02, RX72N HW
00294     manual R01UH0824EJ0111 Table 23.6) */
00295     REG8_AT(0x0008C140U + 2U * 8U + 5U) = 0x02U; /* P25 MTCLKB -- enc0 B */
00296     REG8_AT(0x0008C140U + 10U * 8U + 1U) = 0x02U; /* PA1 MTCLKC -- enc1 A */
00297     REG8_AT(0x0008C140U + 12U * 8U + 5U) = 0x02U; /* PC5 MTCLKD -- enc1 B */
00298     REG8_AT(0x0008C140U + 12U * 8U + 2U) = 0x03U; /* PC2 TCLKA -- enc3 A (TPU1, reads BL per harness) */
00299     REG8_AT(0x0008C140U + 10U * 8U + 3U) = 0x04U; /* PA3 TCLKB -- enc3 B (TPU1, reads BL per harness) */
00300     REG8_AT(0x0008C140U + 12U * 8U + 0U) = 0x03U; /* PC0 TCLKC -- enc2 A (TPU2, reads BR per harness) */
00301     REG8_AT(0x0008C140U + 11U * 8U + 3U) = 0x04U; /* PB3 TCLKD -- enc2 B (TPU2, reads BR per harness) */
00302     *pwpr = 0x00U; *pwpr = 0x80U;
```

```

00302
00303 /* set PMR=1 to route the pad to the peripheral */
00304 *p2pmr |= (uint8_t)((1U « 4) | (1U « 5));
00305 *papmr |= (uint8_t)((1U « 1) | (1U « 3));
00306 *pcpmr |= (uint8_t)((1U « 0) | (1U « 2) | (1U « 5));
00307 *pbpmr |= (uint8_t)(1U « 3);
00308
00309 /* Step 4: phase counting mode 1 on MTU1 + MTU2 via accessors. */
00310 volatile rx_mtu_channel_regs_t* m1 = mtu1();
00311 m1->tcr = 0;
00312 m1->tmdr = 0x04U; /* phase counting mode 1 */
00313 m1->tiorh = 0;
00314 m1->tiorl = 0;
00315 m1->tier = 0;
00316 m1->tsr = 0;
00317 m1->tcnt = 0;
00318 volatile rx_mtu_channel_regs_t* m2 = mtu2();
00319 m2->tcr = 0;
00320 m2->tmdr = 0x04U;
00321 m2->tiorh = 0;
00322 m2->tiorl = 0;
00323 m2->tier = 0;
00324 m2->tsr = 0;
00325 m2->tcnt = 0;
00326
00327 /* Step 5: configure TPU1 + TPU2 for phase counting mode 1 via accessors. */
00328 volatile rx_tpu_regs_t* t1 = tpu1();
00329 t1->tcr = 0;
00330 t1->tmdr = 0x04U;
00331 t1->tcnt = 0;
00332 volatile rx_tpu_regs_t* t2 = tpu2();
00333 t2->tcr = 0;
00334 t2->tmdr = 0x04U;
00335 t2->tcnt = 0;
00336
00337 /* Step 6: disable noise filter on TPU1/TPU2 explicitly (HW reset clears,
00338  * but ensure leftover state from a prior run is gone). */
00339 tpu_control()->nfcr[1] = 0;
00340 tpu_control()->nfcr[2] = 0;
00341
00342 /* Step 7: start counters. */
00343 mtu_tstra()->tstr |= (uint8_t)(k_mtu_tstr_cst1 | k_mtu_tstr_cst2);
00344 tpu_control()->tstr |= (uint8_t)(k_tpu_tstr_cst1 | k_tpu_tstr_cst2);
00345
00346 return true;
00347 }

```

References [REG8_AT](#).

Referenced by [main\(\)](#).

Here is the caller graph for this function:



4.5.5.5 error_park()

```

void error_park (
    const char * msg) [static]

```

Definition at line 380 of file [main.c](#).

```

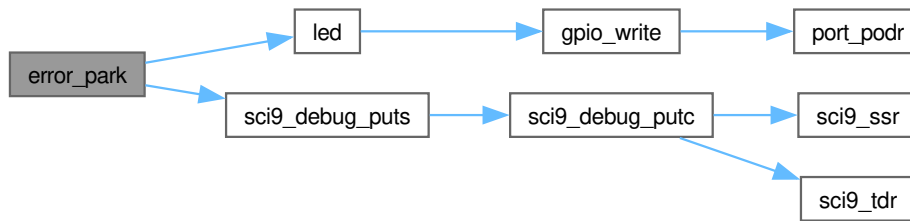
00381 {
00382     sci9_debug_puts("FATAL ");
00383     sci9_debug_puts(msg);
00384     sci9_debug_puts("\n");
00385     led(k_port_b, k_bit_led5, true);
00386     for (;;) { __asm__ volatile("nop"); }
00387 }

```

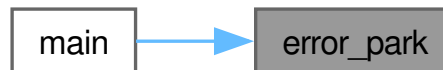
References [k_bit_led5](#), [k_port_b](#), [led\(\)](#), and [sci9_debug_puts\(\)](#).

Referenced by [main\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.5.6 gpio_set_output()

```

void gpio_set_output (
    uint8_t port,
    uint8_t pin)  [inline], [static]
  
```

Definition at line 80 of file [main.c](#).

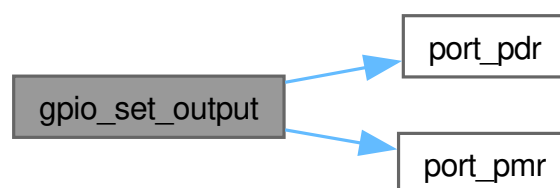
```

00081 {
00082  *port_pmr(port) &= (uint8_t) ~(1U « pin);
00083  *port_pdr(port) |= (uint8_t)(1U « pin);
00084 }
  
```

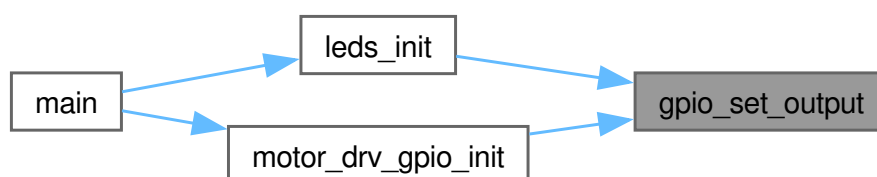
References [port_pdr\(\)](#), and [port_pmr\(\)](#).

Referenced by [leds_init\(\)](#), and [motor_drv_gpio_init\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.5.7 gpio_write()

```
void gpio_write (
    uint8_t port,
    uint8_t pin,
    bool high)  [inline], [static]
```

Definition at line 85 of file [main.c](#).

```
00086 {
00087     if (high) { *port_podr(port) |= (uint8_t)(1U « pin); }
00088     else      { *port_podr(port) &= (uint8_t) ~(1U « pin); }
00089 }
```

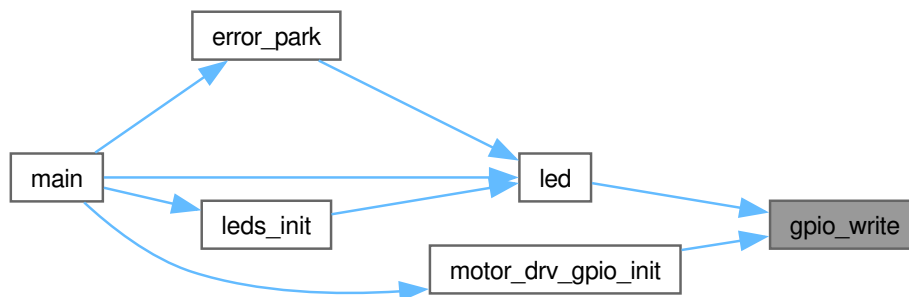
References [port_podr\(\)](#).

Referenced by [led\(\)](#), and [motor_drv_gpio_init\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.5.8 led()

```
void led (
    uint8_t port,
    uint8_t bit,
    bool on)  [inline], [static]
```

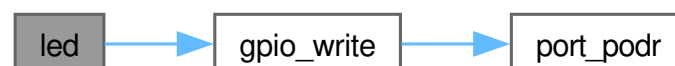
Definition at line 90 of file [main.c](#).

```
00090 { gpio_write(port, bit, on); }
```

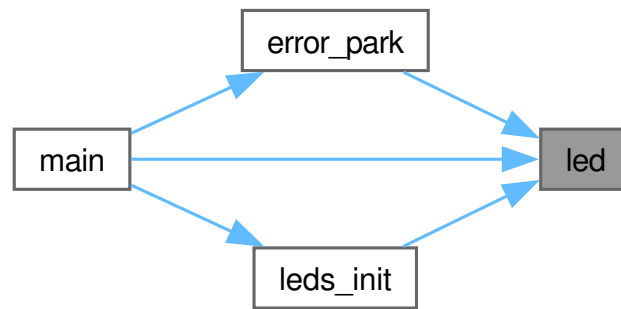
References [gpio_write\(\)](#).

Referenced by [error_park\(\)](#), [leds_init\(\)](#), and [main\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.5.9 leds_init()

```
void leds_init (
    void ) [static]
```

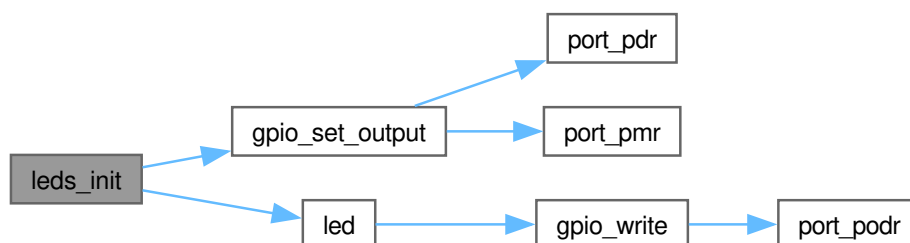
Definition at line 92 of file [main.c](#).

```
00093 {
00094   gpio_set_output(k_port_a, k_bit_led0); led(k_port_a, k_bit_led0, false);
00095   gpio_set_output(k_port_b, k_bit_led1); led(k_port_b, k_bit_led1, false);
00096   gpio_set_output(k_port_7, k_bit_led2); led(k_port_7, k_bit_led2, false);
00097   gpio_set_output(k_port_7, k_bit_led3); led(k_port_7, k_bit_led3, false);
00098   gpio_set_output(k_port_b, k_bit_led4); led(k_port_b, k_bit_led4, false);
00099   gpio_set_output(k_port_b, k_bit_led5); led(k_port_b, k_bit_led5, false);
00100 }
```

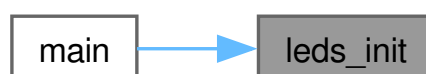
References [gpio_set_output\(\)](#), [k_bit_led0](#), [k_bit_led1](#), [k_bit_led2](#), [k_bit_led3](#), [k_bit_led4](#), [k_bit_led5](#), [k_port_7](#), [k_port_a](#), [k_port_b](#), and [led\(\)](#).

Referenced by [main\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.5.10 main()

```
int main (
    void )
```

Definition at line 398 of file [main.c](#).

```
00399 {
00400     clock_init();
00401     leds_init();
00402     led(k_port_b, k_bit_led1, true);
00403
00404     sci9_debug_init();
00405     motor_drv_gpio_init();
00406
00407     static rx_motor_handle_t s_motors[k_motor_count];
00408     if (!motor_pwm_init(s_motors)) { error_park("motor_pwm_init"); }
00409     if (!encoders_init()) { error_park("encoders_init"); }
00410
00411     /* === Sanity check: can MTU1 count AT ALL using its internal clock?
00412      * This bypasses the encoder pin routing entirely. Set TCR.TPSC=000
00413      * (PCLK/1) and TMDR=0 (normal mode). At 60 MHz PCLKB the counter should
00414      * wrap nearly continuously. If TCNT stays at 0x0000 here too, MTU1
00415      * itself isn't running -- not a pin routing issue. */
00416     REG8_AT(0x000C1280U) &= (uint8_t)~(uint8_t)((1U << 1) | (1U << 2)); /* stop MTU1, MTU2 */
00417     REG8_AT(0x000C1380U) = 0x00U; /* TCR = TPSC=000 (PCLK/1), CKEG=0, CCLR=0 */
00418     REG8_AT(0x000C1381U) = 0x00U; /* TMDR = normal mode */
00419     REG16_AT(0x000C1386U) = 0; /* TCNT = 0 */
00420     REG8_AT(0x000C1280U) |= (uint8_t)(1U << 1); /* start MTU1 */
00421     busy_wait_ms(10);
00422     uint16_t internal_cnt = REG16_AT(0x000C1386U);
00423     sci9_debug_puts("MTU1 internal-clock test (10 ms PCLK/1): TCNT=0x");
00424     sci9_debug_uthex16(internal_cnt);
00425     sci9_debug_puts(internal_cnt != 0 ? " ALIVE\n" : " DEAD\n");
00426     /* Restore MTU1+MTU2 to phase counting mode for the spin loop. */
00427     REG8_AT(0x000C1280U) &= (uint8_t)~(uint8_t)((1U << 1) | (1U << 2));
00428     REG8_AT(0x000C1380U) = 0x00U;
00429     REG8_AT(0x000C1381U) = 0x04U;
00430     REG16_AT(0x000C1386U) = 0;
00431     REG8_AT(0x000C1400U) = 0x00U;
00432     REG8_AT(0x000C1401U) = 0x04U;
00433     REG16_AT(0x000C1406U) = 0;
00434     REG8_AT(0x000C1280U) |= (uint8_t)((1U << 1) | (1U << 2));
00435
00436     /* Loop the test forever so any capture window catches a full cycle.
00437      * Each iteration: print banner, sample, spin 3s, sample, stop, summarize,
00438      * idle 2s, repeat. */
00439     uint16_t iter = 0;
00440     for (;;) {
00441         sci9_debug_puts("\n=== ENCODER_TEST iter=");
00442         sci9_debug_uthex16(iter);
00443         sci9_debug_puts("===\n");
00444
00445         uint16_t before[k_motor_count] = {0};
00446         for (uint8_t i = 0; i < k_motor_count; i++) {
00447             if (!encoder_read_raw(i, &before[i])) { error_park("encoder_read_raw before"); }
00448         }
00449         print_counts("before", before);
00450
00451         /* Dump MTU1 + TSTRA + a few port registers using production accessors. */
00452         {
00453             volatile rx_mtu_channel_regs_t* m = mtu1();
00454             sci9_debug_puts("regs: TCR="); sci9_debug_uthex16(m->tcr);
00455             sci9_debug_puts(" TMDR="); sci9_debug_uthex16(m->tmdr);
00456             sci9_debug_puts(" TIORH="); sci9_debug_uthex16(m->tiorh);
00457             sci9_debug_puts(" TIER="); sci9_debug_uthex16(m->tier);
00458             sci9_debug_puts(" TSR="); sci9_debug_uthex16(m->tsr);
00459             sci9_debug_puts(" TCNT="); sci9_debug_uthex16(m->tcnt);
00460             sci9_debug_puts(" TSTRA="); sci9_debug_uthex16(mtu_tstra()->tstr);
00461             sci9_debug_puts(" P24PFS="); sci9_debug_uthex16(*(volatile uint8_t*)0x0008C154U);
00462             sci9_debug_puts(" P25PFS="); sci9_debug_uthex16(*(volatile uint8_t*)0x0008C155U);
00463             sci9_debug_puts(" P2PMR="); sci9_debug_uthex16(*(volatile uint8_t*)0x0008C062U);
00464             sci9_debug_puts(" P2PIDR="); sci9_debug_uthex16(*(volatile uint8_t*)0x0008C042U);
00465             sci9_debug_puts("\n");
00466         }
00467         sci9_debug_puts("spinning motors (per-motor sign applied) for 3s, sampling each second\n");
00468         led(k_port_7, k_bit_led2, true);
00469         for (uint8_t i = 0; i < k_motor_count; i++) {
00470             const float duty = 50.0F * (float)k_motors[i].direction_sign;
00471             (void)rx_motor_set_duty(&s_motors[i], duty);
00472         }
00473
00474         /* Burst-read MTU1.TCNT in a tight loop right after zeroing it -- if
00475          * MTU is counting fast (TSR.TCFV is set in our dump => overflow has
00476          * fired at least once), the values will move quickly across this loop,
```

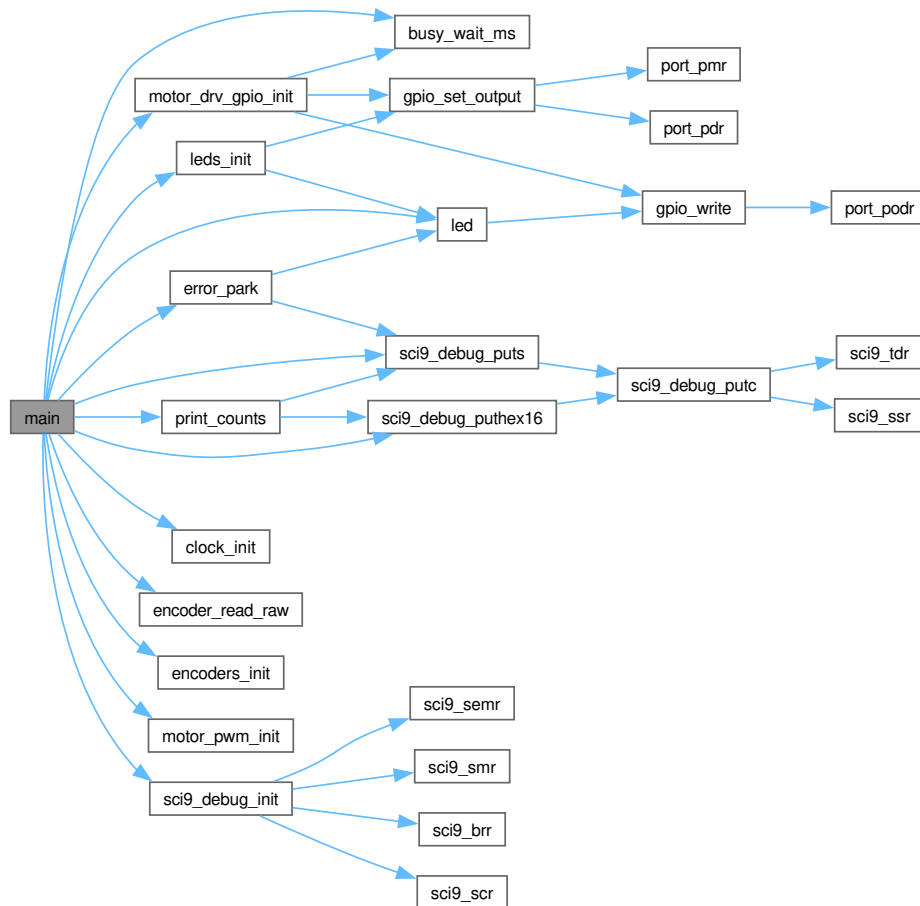
```

00477      * even though the periodic 1-second sample lands at 0. */
00478      mtu1()->tcnt = 0;
00479      sci9_debug_puts("burst MTU1.TCNT:");
00480      for (uint8_t s = 0; s < 8; s++) {
00481          busy_wait_ms(1);
00482          uint16_t c = mtu1()->tcnt;
00483          sci9_debug_puts(" ");
00484          sci9_debug_puthex16(c);
00485      }
00486      sci9_debug_puts("\n");
00487
00488      /* Sample 3 times during the spin so we can see if counters move at all. */
00489      volatile uint8_t *p2_pidr = (volatile uint8_t *)0x0008C042U;
00490      volatile uint8_t *pa_pidr = (volatile uint8_t *)0x0008C044U;
00491      volatile uint8_t *pc_pidr = (volatile uint8_t *)0x0008C04CU;
00492      uint16_t mid[k_motor_count] = {0};
00493      for (uint8_t s = 0; s < 3; s++) {
00494          busy_wait_ms(k_spin_ms / 3U);
00495          for (uint8_t i = 0; i < k_motor_count; i++) {
00496              (void)encoder_read_raw(i, &mid[i]);
00497          }
00498          sci9_debug_puts("mid ");
00499          sci9_debug_puthex16(mid[0]); sci9_debug_puts(" ");
00500          sci9_debug_puthex16(mid[1]); sci9_debug_puts(" ");
00501          sci9_debug_puthex16(mid[2]); sci9_debug_puts(" ");
00502          sci9_debug_puthex16(mid[3]);
00503          sci9_debug_puts(" pidr2="); sci9_debug_puthex16(*p2_pidr);
00504          sci9_debug_puts(" pidrA="); sci9_debug_puthex16(*pa_pidr);
00505          sci9_debug_puts(" pidrC="); sci9_debug_puthex16(*pc_pidr);
00506          sci9_debug_puts("\n");
00507      }
00508
00509      uint16_t after[k_motor_count] = {0};
00510      for (uint8_t i = 0; i < k_motor_count; i++) {
00511          if (!encoder_read_raw(i, &after[i])) { error_park("encoder_read_raw after"); }
00512      }
00513      print_counts("after ", after);
00514
00515      for (uint8_t i = 0; i < k_motor_count; i++) {
00516          (void)rx_motor_set_duty(&s_motors[i], 0.0F);
00517      }
00518      led(k_port_7, k_bit_led2, false);
00519      sci9_debug_puts("motors stopped\n");
00520      busy_wait_ms(k_settle_ms);
00521
00522      uint16_t delta[k_motor_count];
00523      for (uint8_t i = 0; i < k_motor_count; i++) {
00524          delta[i] = (uint16_t)(after[i] - before[i]);
00525      }
00526      print_counts("delta ", delta);
00527
00528      bool pass = true;
00529      for (uint8_t i = 0; i < k_motor_count; i++) {
00530          int32_t signed_delta = (int32_t)(int16_t)delta[i];
00531          uint32_t mag = (uint32_t)((signed_delta < 0) ? -signed_delta : signed_delta);
00532          if (mag < k_min_delta) {
00533              pass = false;
00534              sci9_debug_puts("encoder FAIL idx=");
00535              sci9_debug_puthex16((uint16_t)i);
00536              sci9_debug_puts("\n");
00537          }
00538      }
00539      sci9_debug_puts(pass ? "summary PASS\n" : "summary FAIL\n");
00540      if (pass) { led(k_port_b, k_bit_led4, true); }
00541      else { led(k_port_b, k_bit_led5, true); }
00542
00543      /* Idle 2s then repeat. */
00544      busy_wait_ms(2000);
00545      iter++;
00546  }
00547 }

```

References [busy_wait_ms\(\)](#), [clock_init\(\)](#), [encoder_read_raw\(\)](#), [encoders_init\(\)](#), [error_park\(\)](#), [k_bit_led1](#), [k_bit_led2](#), [k_bit_led4](#), [k_bit_led5](#), [k_min_delta](#), [k_motor_count](#), [k_motors](#), [k_port_7](#), [k_port_b](#), [k_settle_ms](#), [k_spin_ms](#), [led\(\)](#), [leds_init\(\)](#), [motor_drv_gpio_init\(\)](#), [motor_pwm_init\(\)](#), [print_counts\(\)](#), [REG16_AT](#), [REG8_AT](#), [sci9_debug_init\(\)](#), [sci9_debug_puthex16\(\)](#), and [sci9_debug_puts\(\)](#).

Here is the call graph for this function:



4.5.5.11 motor_drv_gpio_init()

```
void motor_drv_gpio_init (
    void ) [static]
```

Definition at line 183 of file [main.c](#).

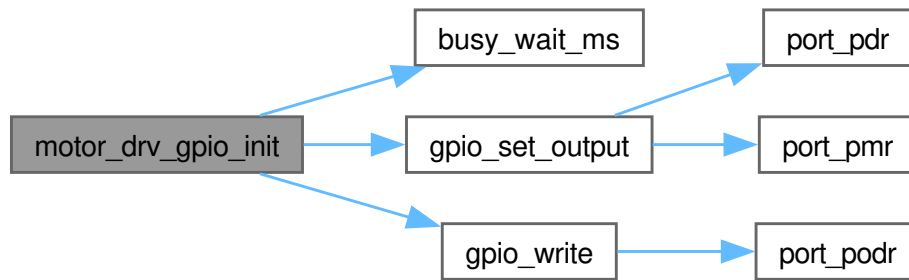
```

00184 {
00185     for (uint8_t i = 0; i < k_motor_count; i++) {
00186         gpio_set_output(k_motors[i].drvoff_port, k_motors[i].drvoff_pin);
00187         gpio_write(k_motors[i].drvoff_port, k_motors[i].drvoff_pin, true);
00188     }
00189     for (uint8_t i = 0; i < k_motor_count; i++) {
00190         gpio_set_output(k_motors[i].nsleep_port, k_motors[i].nsleep_pin);
00191         gpio_write(k_motors[i].nsleep_port, k_motors[i].nsleep_pin, true);
00192     }
00193     busy_wait_ms(10); /* tWAKE */
00194     for (uint8_t i = 0; i < k_motor_count; i++) {
00195         gpio_write(k_motors[i].drvoff_port, k_motors[i].drvoff_pin, false);
00196     }
00197 }
```

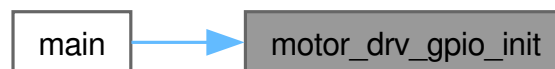
References [busy_wait_ms\(\)](#), [gpio_set_output\(\)](#), [gpio_write\(\)](#), [k_motor_count](#), and [k_motors](#).

Referenced by [main\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.5.12 motor_pwm_init()

```
bool motor_pwm_init (
    rx_motor_handle_t handles[k_motor_count]) [static]
```

Definition at line 202 of file [main.c](#).

```
00203 {
00204     for (uint8_t i = 0; i < k_motor_count; i++) {
00205         const rx_motor_config_t cfg = {
00206             .channel    = k_motors[i].gptw_channel,
00207             .output_a    = k_gptw_output_a,
00208             .output_b    = k_gptw_output_b,
00209             .pwm_freq_hz = 20000U,
00210             .dead_time_ns = 1000U,
00211             .invert_pwm  = false,
00212             .port_a_idx  = (uint8_t)rx_port_from_pin(k_motors[i].in2_pin),
00213             .bit_a       = (uint8_t)rx_pin_from_pin(k_motors[i].in2_pin),
00214             .port_b_idx  = (uint8_t)rx_port_from_pin(k_motors[i].in1_pin),
00215             .bit_b       = (uint8_t)rx_pin_from_pin(k_motors[i].in1_pin),
00216         };
00217         if (rx_motor_init(&handles[i], &cfg) != k_rx_ok) { return false; }
00218     }
00219     return true;
00220 }
```

References [k_motor_count](#), and [k_motors](#).

Referenced by [main\(\)](#).

Here is the caller graph for this function:



4.5.5.13 port_pdr()

```
volatile uint8_t * port_pdr (
    uint8_t port)  [inline], [static]
```

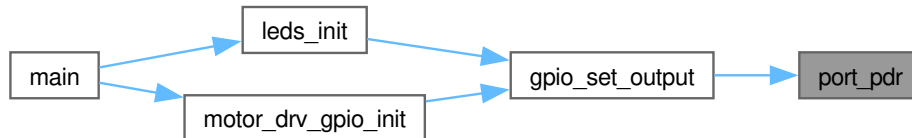
Definition at line 61 of file [main.c](#).

```
00061 { return &REG8(k_port_pdr_base + port); }
```

References [k_port_pdr_base](#), and [REG8](#).

Referenced by [gpio_set_output\(\)](#).

Here is the caller graph for this function:



4.5.5.14 port_pmr()

```
volatile uint8_t * port_pmr (
    uint8_t port)  [inline], [static]
```

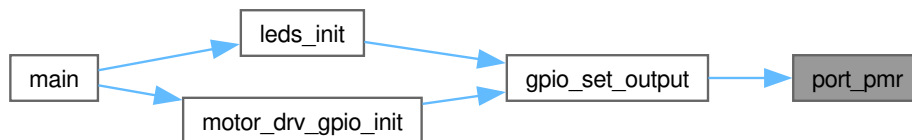
Definition at line 63 of file [main.c](#).

```
00063 { return &REG8(k_port_pmr_base + port); }
```

References [k_port_pmr_base](#), and [REG8](#).

Referenced by [gpio_set_output\(\)](#).

Here is the caller graph for this function:



4.5.5.15 port_podr()

```
volatile uint8_t * port_podr (
    uint8_t port)  [inline], [static]
```

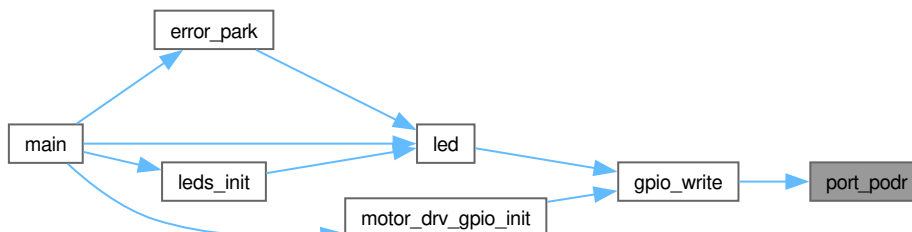
Definition at line 62 of file [main.c](#).

```
00062 { return &REG8(k_port_podr_base + port); }
```

References [k_port_podr_base](#), and [REG8](#).

Referenced by [gpio_write\(\)](#).

Here is the caller graph for this function:



4.5.5.16 print_counts()

```
void print_counts (
    const char * label,
    uint16_t c[k_motor_count]) [static]
```

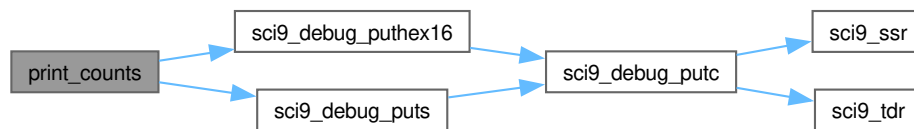
Definition at line 366 of file [main.c](#).

```
00367 {
00368     sci9_debug_puts(label);
00369     sci9_debug_puts(" m0=");
00370     sci9_debug_puthex16(c[0]);
00371     sci9_debug_puts(" m1=");
00372     sci9_debug_puthex16(c[1]);
00373     sci9_debug_puts(" m2=");
00374     sci9_debug_puthex16(c[2]);
00375     sci9_debug_puts(" m3=");
00376     sci9_debug_puthex16(c[3]);
00377     sci9_debug_puts("\n");
00378 }
```

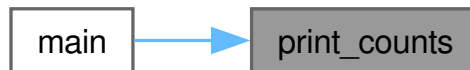
References [k_motor_count](#), [sci9_debug_puthex16\(\)](#), and [sci9_debug_puts\(\)](#).

Referenced by [main\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.5.17 sci9_debug_init()

```
void sci9_debug_init (
    void ) [extern]
```

Definition at line 104 of file [sci9.c](#).

```
00105 {
00106     /* Step 1: release SCI9 from module stop. SCI9 = MSTPCRC bit 26 per
00107      * RX72N HW manual page 410 (NOT MSTPCRB.bit22 -- that's PDC). */
00108     *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00109     system_regs()->mstpcrc &= ~(uint32_t)(1UL « (uint32_t)k_mstpc_sci9_bit);
00110     *prcr_reg() = k_rx_prcr_lock;
00111
00112     /* Step 2: configure PB6 (RXD9) and PB7 (TXD9) via MPC. Use the struct
00113      * accessor so the compiler computes the correct PFS offsets via
00114      * static_assert-checked layout. */
00115     mpc()->pwpr = k_pwpr_unlock_b0; /* clear B0WI */
00116     mpc()->pwpr = k_pwpr_pfswe; /* allow PFS writes */
00117     mpc()->pb6pfs = k_psel_sci9;
00118     mpc()->pb7pfs = k_psel_sci9;
00119     mpc()->pwpr = k_pwpr_unlock_b0; /* clear PFSWE */
00120     mpc()->pwpr = k_pwpr_b0wi; /* re-protect */
00121
00122     /* Step 3: PDR before PMR (production uart.c does this). PB7 = output
00123      * for TX, PB6 = input for RX. Then flip PMR to peripheral mode. */
00124     portb()->pdr |= (uint8_t)k_pb7_mask;
```

```

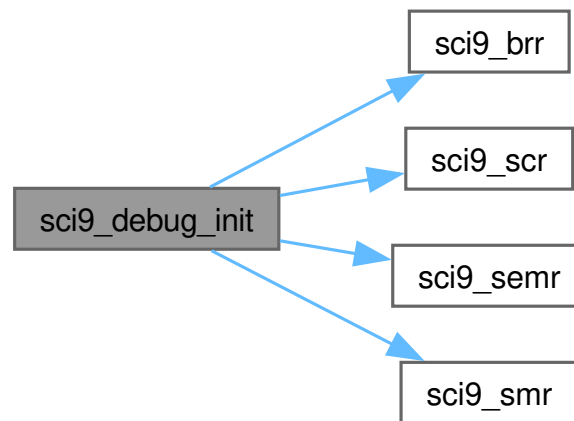
00125 portb()->pdr &= (uint8_t) ~k_pb6_mask;
00126 portb()->pmr |= (uint8_t)(k_pb6_mask | k_pb7_mask);
00127
00128 /* Step 4: SCI9 init. Match the production order:
00129  *   SCR=0 -> SMR -> BRR -> settle -> SCR=TE|RE -> SEMR.
00130  *   Don't touch SCMR (reset default 0xF2 is correct). */
00131 *sci9_scr() = k_scr_disabled;
00132 *sci9_smr() = k_smr_8n1_async;
00133 *sci9_brr() = k_brr_115200;
00134
00135 /* Wait roughly one bit time before enabling TX/RX. */
00136 for (volatile uint32_t i = 0U; i < (uint32_t)k_brr_settle_loops; i++) {
00137     __asm__ volatile("nop");
00138 }
00139
00140 *sci9_scr() = k_scr_txrx_enabled;
00141 *sci9_semr() = k_semr_default;
00142 }

```

References [k_brr_115200](#), [k_brr_settle_loops](#), [k_mstpc_sci9_bit](#), [k_pb6_mask](#), [k_pb7_mask](#), [k_psel_sci9](#), [k_pwpr_b0wi](#), [k_pwpr_pfswe](#), [k_pwpr_unlock_b0](#), [k_scr_disabled](#), [k_scr_txrx_enabled](#), [k_semr_default](#), [k_smr_8n1_async](#), [sci9_brr\(\)](#), [sci9_scr\(\)](#), [sci9_semr\(\)](#), and [sci9_smr\(\)](#).

Referenced by [main\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.5.18 sci9_debug_puthex16()

```

void sci9_debug_puthex16 (
    uint16_t v) [extern]

```

Definition at line 179 of file [sci9.c](#).

```

00180 {
00181     static const char hex_digits[] = "0123456789ABCDEF";
00182     sci9_debug_putc('0');
00183     sci9_debug_putc('x');
00184     for (int8_t shift = (int8_t)k_hex_shift_init16; shift >= 0;
00185          shift -= (int8_t)k_hex_shift_step) {
00186         sci9_debug_putc(hex_digits[(v » (uint8_t)shift) & (uint8_t)k_hex_nibble_mask]);
00187     }
00188 }

```

References [k_hex_nibble_mask](#), [k_hex_shift_init16](#), [k_hex_shift_step](#), and [sci9_debug_putc\(\)](#).

Referenced by [main\(\)](#), and [print_counts\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.5.19 sci9_debug_puts()

```
void sci9_debug_puts (
    const char * s) [extern]
```

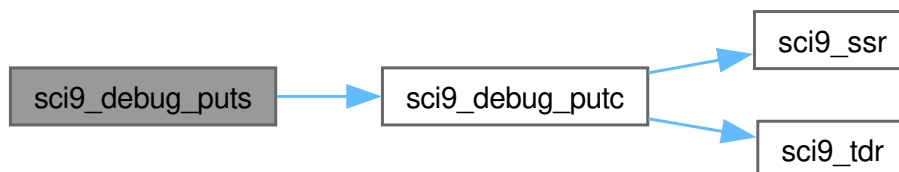
Definition at line 155 of file [sci9.c](#).

```
00156 {
00157     if (s == (const char *)0) {
00158         return;
00159     }
00160     while (*s != '\0') {
00161         if (*s == '\n') {
00162             sci9_debug_putc('\r');
00163         }
00164         sci9_debug_putc(*s++);
00165     }
00166 }
```

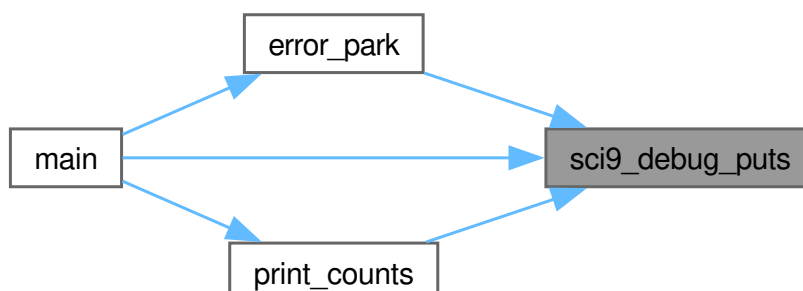
References [sci9_debug_putc\(\)](#).

Referenced by [error_park\(\)](#), [main\(\)](#), and [print_counts\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.6 Variable Documentation

4.5.6.1 k_enc_pins

const [enc_pin_t](#) k_enc_pins[] [static]

Initial value:

```
= {
  {2, 4, false}, {2, 5, false},
  {10, 1, false}, {12, 5, false},
  {12, 0, true}, {11, 3, true},
  {12, 2, true}, {10, 3, true},
}
```

Definition at line 238 of file [main.c](#).

```
00238 {
00239 {2, 4, false}, {2, 5, false}, /* enc0: P24 / P25 (MTU1 MTCLKA/B) -- FL */
00240 {10, 1, false}, {12, 5, false}, /* enc1: PA1 / PC5 (MTU2 MTCLKC/D) -- FR */
00241 {12, 0, true}, {11, 3, true}, /* enc2: PC0 / PB3 (TPU2 TCLKC/D, harness) -- BR */
00242 {12, 2, true}, {10, 3, true}, /* enc3: PC2 / PA3 (TPU1 TCLKA/B, harness) -- BL */
00243 };
```

4.5.6.2 k_motors

const [motor_wiring_t](#) k_motors[k_motor_count] [static]

Initial value:

```
= {
  { .gptw_channel = k_gptw_channel_0, .in2_pin = k_rx_p2_3, .in1_pin = k_rx_p1_7,
    .drvoff_port = k_port_6, .drvoff_pin = 1, .nsleep_port = k_port_6, .nsleep_pin = 0,
    .is_tpu = false, .timer_channel = 1, .direction_sign = -1},

  { .gptw_channel = k_gptw_channel_1, .in2_pin = k_rx_p2_2, .in1_pin = k_rx_pc_3,
    .drvoff_port = k_port_6, .drvoff_pin = 3, .nsleep_port = k_port_6, .nsleep_pin = 2,
    .is_tpu = false, .timer_channel = 2, .direction_sign = +1},

  { .gptw_channel = k_gptw_channel_2, .in2_pin = k_rx_pe_3, .in1_pin = k_rx_p8_6,
    .drvoff_port = k_port_e, .drvoff_pin = 0, .nsleep_port = k_port_6, .nsleep_pin = 4,
    .is_tpu = true, .timer_channel = 2, .direction_sign = +1},

  { .gptw_channel = k_gptw_channel_3, .in2_pin = k_rx_pe_7, .in1_pin = k_rx_pc_6,
    .drvoff_port = k_port_e, .drvoff_pin = 2, .nsleep_port = k_port_e, .nsleep_pin = 1,
    .is_tpu = true, .timer_channel = 1, .direction_sign = -1},
}
```

Definition at line 141 of file [main.c](#).

```
00141 {
00142 /* M0 = FL (front-left). Wired backwards. */
00143 { .gptw_channel = k_gptw_channel_0, .in2_pin = k_rx_p2_3, .in1_pin = k_rx_p1_7,
00144   .drvoff_port = k_port_6, .drvoff_pin = 1, .nsleep_port = k_port_6, .nsleep_pin = 0,
00145   .is_tpu = false, .timer_channel = 1, .direction_sign = -1},
00146 /* M1 = FR. */
00147 { .gptw_channel = k_gptw_channel_1, .in2_pin = k_rx_p2_2, .in1_pin = k_rx_pc_3,
00148   .drvoff_port = k_port_6, .drvoff_pin = 3, .nsleep_port = k_port_6, .nsleep_pin = 2,
00149   .is_tpu = false, .timer_channel = 2, .direction_sign = +1},
00150 /* M2 = BR. Harness: Hall cable lands on TPU2 pins (PC0/PB3). */
00151 { .gptw_channel = k_gptw_channel_2, .in2_pin = k_rx_pe_3, .in1_pin = k_rx_p8_6,
00152   .drvoff_port = k_port_e, .drvoff_pin = 0, .nsleep_port = k_port_6, .nsleep_pin = 4,
00153   .is_tpu = true, .timer_channel = 2, .direction_sign = +1},
00154 /* M3 = BL. Wired backwards (matches FL on left side). Harness: Hall cable
00155  * lands on TPU1 pins (PC2/PA3). */
00156 { .gptw_channel = k_gptw_channel_3, .in2_pin = k_rx_pe_7, .in1_pin = k_rx_pc_6,
00157   .drvoff_port = k_port_e, .drvoff_pin = 2, .nsleep_port = k_port_e, .nsleep_pin = 1,
00158   .is_tpu = true, .timer_channel = 1, .direction_sign = -1},
00159 };
```

Referenced by [main\(\)](#), [motor_drv_gpio_init\(\)](#), and [motor_pwm_init\(\)](#).

4.6 main.c

[Go to the documentation of this file.](#)

```

00001 /**
00002  * @file main.c
00003  * @brief Closed-loop encoder verification test.
00004  *
00005  * @details
00006  * Drives all 4 motors at +50 percent duty for ~3 seconds while sampling each
00007  * encoder's hardware count. Stops motors, then dumps the count deltas via
00008  * SCI9 UART so the host (cat /dev/ttyACM0 @ 115200) can capture and verify
00009  * each encoder produced ticks.
00010  *
00011  * Expected output on /dev/ttyACM0:
00012  * ENCODER_TEST start
00013  * init clock ok
00014  * init leds ok
00015  * init uart ok
00016  * init drv gpio ok
00017  * init pwm ok
00018  * init enc ok
00019  * before m0=0xFFFF m1=0xFFFF m2=0xFFFF m3=0xFFFF
00020  * spinning motors at +50pct for 3 seconds
00021  * after m0=0xFFFF m1=0xFFFF m2=0xFFFF m3=0xFFFF
00022  * motors stopped
00023  * delta m0=0xFFFF m1=0xFFFF m2=0xFFFF m3=0xFFFF
00024  * summary FAIL m0,m3 (or PASS)
00025  *
00026  * SPDX-License-Identifier: MIT
00027  * @copyright Copyright (c) 2026 Locked Inc.
00028  */
00029
00030 #include <stdint.h>
00031 #include <stdbool.h>
00032 #include <stddef.h>
00033
00034 #include "rx_motor.h"
00035 #include "rx_gptw.h"
00036 #include "rx_mpc.h"
00037 #include "rx_port_constants.h"
00038 #include "rx_mtu_encoder.h"
00039 #include "rx_encoder_tpu.h"
00040 #include "rx72n_mtu_regs.h"
00041 #include "rx72n_tpu_regs.h"
00042 #include "rx72n_system_regs.h"
00043 #include "rx_register_protection.h"
00044
00045 extern void clock_init(void);
00046 extern void sci9_debug_init(void);
00047 extern void sci9_debug_puts(const char *s);
00048 extern void sci9_debug_puthex16(uint16_t v);
00049
00050 /**
=====
00051  * Direct port-register access (same pattern as motor_spin_test).
00052  *
=====
00053  */
00053 #define REG8(a) (*(volatile uint8_t *) (uintptr_t)(a))
00054
00055 typedef enum : uintptr_t {
00056     k_port_pdr_base = 0x0008C000U,
00057     k_port_podr_base = 0x0008C020U,
00058     k_port_pmr_base = 0x0008C060U,
00059 } port_reg_base_t;
00060
00061 static inline volatile uint8_t *port_pdr(uint8_t port) { return &REG8(k_port_pdr_base + port); }
00062 static inline volatile uint8_t *port_podr(uint8_t port) { return &REG8(k_port_podr_base + port); }
00063 static inline volatile uint8_t *port_pmr(uint8_t port) { return &REG8(k_port_pmr_base + port); }
00064
00065 /**
=====
00066  * Status LEDs (active-high, same map as motor_spin_test).
00067  *
=====
00068  */
00068 typedef enum : uint8_t {
00069     k_port_a = 10,
00070     k_port_b = 11,
00071     k_port_7 = 7,
00072     k_bit_led0 = 7, /* PA7 D9 -- heartbeat */
00073     k_bit_led1 = 0, /* PB0 D10 -- after init */
00074     k_bit_led2 = 1, /* P71 D11 -- motor PWM running */
00075     k_bit_led3 = 2, /* P72 D12 -- encoders all ticked */
00076     k_bit_led4 = 1, /* PB1 D13 -- test PASS */

```

```

00077 k_bit_led5 = 2, /* PB2 D14 -- test FAIL or init error */
00078 } status_led_pins_t;
00079
00080 static inline void gpio_set_output(uint8_t port, uint8_t pin)
00081 {
00082     *port_pmr(port) &= (uint8_t) ~(1U « pin);
00083     *port_pdr(port) |= (uint8_t)(1U « pin);
00084 }
00085 static inline void gpio_write(uint8_t port, uint8_t pin, bool high)
00086 {
00087     if (high) { *port_podr(port) |= (uint8_t)(1U « pin); }
00088     else { *port_podr(port) &= (uint8_t) ~(1U « pin); }
00089 }
00090 static inline void led(uint8_t port, uint8_t bit, bool on) { gpio_write(port, bit, on); }
00091
00092 static void leds_init(void)
00093 {
00094     gpio_set_output(k_port_a, k_bit_led0); led(k_port_a, k_bit_led0, false);
00095     gpio_set_output(k_port_b, k_bit_led1); led(k_port_b, k_bit_led1, false);
00096     gpio_set_output(k_port_7, k_bit_led2); led(k_port_7, k_bit_led2, false);
00097     gpio_set_output(k_port_7, k_bit_led3); led(k_port_7, k_bit_led3, false);
00098     gpio_set_output(k_port_b, k_bit_led4); led(k_port_b, k_bit_led4, false);
00099     gpio_set_output(k_port_b, k_bit_led5); led(k_port_b, k_bit_led5, false);
00100 }
00101
00102 /*
=====
00103 * Per-motor wiring + per-encoder timer-channel mapping. Mirrors
00104 * src/inc/hardware_config.h but kept local so the test can compile
00105 * without depending on the application include tree.
00106 *
=====
00107 *
00108 * NOTE: The back-wheel Hall encoder harness is cross-wired vs silk. BR's
00109 * encoder cable physically lands on TPU2 pins (PC0/PB3) and BL's lands on
00110 * TPU1 pins (PC2/PA3). The Encoder-unit column below reflects the actual
00111 * hardware wiring, so spinning wheel BR increments enc2 (reading TPU2).
00112 * Matches production (src/tasks/motor_control_task.c:2214-2216).
00113 *
00114 * | Idx | Wheel | GPTW | IN1 (PWM) | IN2 (dir) | DRVOFF | nSLEEP | Encoder unit |
00115 * |-----|-----|-----|-----|-----|-----|-----|-----|
00116 * | 0 | FL | 0 | P17 | P23 | P61 | P60 | MTU1 |
00117 * | 1 | FR | 1 | PC3 | P22 | P63 | P62 | MTU2 |
00118 * | 2 | BR | 2 | P86 | PE3 | PE0 | P64 | TPU2 |
00119 * | 3 | BL | 3 | PC6 | PE7 | PE2 | PE1 | TPU1 |
00120 */
00121 typedef struct {
00122     rx_gptw_channel_t gptw_channel;
00123     rx_port_pin_t in2_pin;
00124     rx_port_pin_t in1_pin;
00125     uint8_t drvoff_port;
00126     uint8_t drvoff_pin;
00127     uint8_t nsleep_port;
00128     uint8_t nsleep_pin;
00129     bool is_tpu; /* false = MTU1/2, true = TPU1/2 */
00130     uint8_t timer_channel; /* MTU 1/2 or TPU 1/2 */
00131     int8_t direction_sign; /* +1 = positive duty drives wheel forward;
00132                             -1 = motor wired backwards (M0 FL, M3 BL) */
00133 } motor_wiring_t;
00134
00135 typedef enum : uint8_t {
00136     k_motor_count = 4,
00137     k_port_6 = 6,
00138     k_port_e = 14,
00139 } test_constants_t;
00140
00141 static const motor_wiring_t k_motors[k_motor_count] = {
00142     /* M0 = FL (front-left). Wired backwards. */
00143     { .gptw_channel = k_gptw_channel_0, .in2_pin = k_rx_p2_3, .in1_pin = k_rx_p1_7,
00144       .drvoff_port = k_port_6, .drvoff_pin = 1, .nsleep_port = k_port_6, .nsleep_pin = 0,
00145       .is_tpu = false, .timer_channel = 1, .direction_sign = -1 },
00146     /* M1 = FR. */
00147     { .gptw_channel = k_gptw_channel_1, .in2_pin = k_rx_p2_2, .in1_pin = k_rx_pc_3,
00148       .drvoff_port = k_port_6, .drvoff_pin = 3, .nsleep_port = k_port_6, .nsleep_pin = 2,
00149       .is_tpu = false, .timer_channel = 2, .direction_sign = +1 },
00150     /* M2 = BR. Harness: Hall cable lands on TPU2 pins (PC0/PB3). */
00151     { .gptw_channel = k_gptw_channel_2, .in2_pin = k_rx_pe_3, .in1_pin = k_rx_p8_6,
00152       .drvoff_port = k_port_e, .drvoff_pin = 0, .nsleep_port = k_port_6, .nsleep_pin = 4,
00153       .is_tpu = true, .timer_channel = 2, .direction_sign = +1 },
00154     /* M3 = BL. Wired backwards (matches FL on left side). Harness: Hall cable
00155        * lands on TPU1 pins (PC2/PA3). */
00156     { .gptw_channel = k_gptw_channel_3, .in2_pin = k_rx_pe_7, .in1_pin = k_rx_pc_6,
00157       .drvoff_port = k_port_e, .drvoff_pin = 2, .nsleep_port = k_port_e, .nsleep_pin = 1,
00158       .is_tpu = true, .timer_channel = 1, .direction_sign = -1 },
00159 };
00160
00161 /*

```

```

=====
00162 * Bounded busy-wait helper (NASA Rule 2). ~1 ms = 240k iters at 240 MHz -O1.
00163 *
=====
00164 */
00165 typedef enum : uint32_t {
00166     /* Empirically calibrated: 40000 iters/ms at 240 MHz with -O1 volatile loop
00167      * (~6 cycles per iteration). 240000 was 6x too slow. */
00168     k_iters_per_ms = 40000U,
00169 } delay_const_t;
00170
00171 static void busy_wait_ms(uint32_t ms)
00172 {
00173     for (uint32_t m = 0; m < ms; m++) {
00174         for (volatile uint32_t i = 0; i < k_iters_per_ms; i++) {
00175             __asm__ volatile("nop");
00176         }
00177     }
00178 }
00179
00180 /*
=====
00181 * DRV8263H DRVOFF/nSLEEP power-up sequence (same as motor_spin_test).
00182 *
=====
00183 */
00184 static void motor_drv_gpio_init(void)
00185 {
00186     for (uint8_t i = 0; i < k_motor_count; i++) {
00187         gpio_set_output(k_motors[i].drvoff_port, k_motors[i].drvoff_pin);
00188         gpio_write(k_motors[i].drvoff_port, k_motors[i].drvoff_pin, true);
00189     }
00190     for (uint8_t i = 0; i < k_motor_count; i++) {
00191         gpio_set_output(k_motors[i].nsleep_port, k_motors[i].nsleep_pin);
00192         gpio_write(k_motors[i].nsleep_port, k_motors[i].nsleep_pin, true);
00193     }
00194     busy_wait_ms(10); /* tWAKE */
00195     for (uint8_t i = 0; i < k_motor_count; i++) {
00196         gpio_write(k_motors[i].drvoff_port, k_motors[i].drvoff_pin, false);
00197     }
00198 }
00199 /*
=====
00200 * GPTW PWM init (per-motor, via the lib).
00201 *
=====
00202 */
00203 static bool motor_pwm_init(rx_motor_handle_t handles[k_motor_count])
00204 {
00205     for (uint8_t i = 0; i < k_motor_count; i++) {
00206         const rx_motor_config_t cfg = {
00207             .channel = k_motors[i].gptw_channel,
00208             .output_a = k_gptw_output_a,
00209             .output_b = k_gptw_output_b,
00210             .pwm_freq_hz = 20000U,
00211             .dead_time_ns = 1000U,
00212             .invert_pwm = false,
00213             .port_a_idx = (uint8_t)rx_port_from_pin(k_motors[i].in2_pin),
00214             .bit_a = (uint8_t)rx_pin_from_pin(k_motors[i].in2_pin),
00215             .port_b_idx = (uint8_t)rx_port_from_pin(k_motors[i].in1_pin),
00216             .bit_b = (uint8_t)rx_pin_from_pin(k_motors[i].in1_pin),
00217         };
00218         if (rx_motor_init(&handles[i], &cfg) != k_rx_ok) { return false; }
00219     }
00220     return true;
00221 }
00222 /*
=====
00223 * Encoder init -- MTU for FL/FR (idx 0, 1), TPU for BR/BL (idx 2, 3).
00224 *
=====
00225 */
00226 typedef enum : uint16_t {
00227     k_counts_per_rev = 1364, /* 341 PPR x 4 (matches production) */
00228 } enc_const_t;
00229
00230 /* Encoder phase A/B input pins. Reflects the cross-wired back-wheel harness:
00231 * BR's Hall cable is on TPU2 pins (PC0/PB3) and BL's is on TPU1 pins
00232 * (PC2/PA3). Matches production (motor_control_task.c:2214-2216).
00233 * Encoder 0 (FL, MTU1): P24 / P25
00234 * Encoder 1 (FR, MTU2): PA1 / PC5
00235 * Encoder 2 (BR, TPU2): PC0 / PB3
00236 * Encoder 3 (BL, TPU1): PC2 / PA3
00237 */
00238 typedef struct { uint8_t port; uint8_t bit; bool is_tpu; } enc_pin_t;

```

```

00238 static const enc_pin_t k_enc_pins[] = {
00239     {2, 4, false}, {2, 5, false}, /* enc0: P24 / P25 (MTU1 MTCLKA/B) -- FL */
00240     {10, 1, false}, {12, 5, false}, /* enc1: PA1 / PC5 (MTU2 MTCLKC/D) -- FR */
00241     {12, 0, true}, {11, 3, true}, /* enc2: PC0 / PB3 (TPU2 TCLKC/D, harness) -- BR */
00242     {12, 2, true}, {10, 3, true}, /* enc3: PC2 / PA3 (TPU1 TCLKA/B, harness) -- BL */
00243 };
00244
00245 /* Direct-register encoder init -- the lib's rx_mpc_set_*_encoder doesn't
00246  * actually write PFS, and rx_mtu_start doesn't actually set TSTRA. Bypass
00247  * both. Verified against RX72N HW manual chapter 25 (MTU3a) + chapter 27
00248  * (TPUa) + chapter 23 (MPC PFS tables 23.4 / 23.6 / 23.10 / 23.14 / 23.16).
00249  */
00250 /* MTU1 (enc0 FL): base 0x000C1380, TCR/TMDR/TCNT at +0/+1/+6, TSTRA bit 1
00251  * MTU2 (enc1 FR): base 0x000C1400, same offsets, TSTRA bit 2
00252  * TPU2 (enc2 BR): base 0x00088130, TCR/TMDR/TCNT at +0/+1/+6, TSTR bit 2 (harness)
00253  * TPU1 (enc3 BL): base 0x00088120, same offsets, TSTR bit 1 (harness)
00254  */
00255 /* Phase counting mode 1 = TMDR=0x04. TCR=0x00 (external clock). PSEL=0x02
00256  * for MTCLK[A-D] on port 2/A/C; TCLK[A-D] on port C/A/C/B.
00257  */
00258
00259 #define REG8_AT(a) (*(volatile uint8_t *) (uintptr_t)(a))
00260 #define REG16_AT(a) (*(volatile uint16_t *) (uintptr_t)(a))
00261 #define REG32_AT(a) (*(volatile uint32_t *) (uintptr_t)(a))
00262
00263 /* MTU/TPU encoder init using the production accessors from
00264  * rx72n_{mtu,tpu,system}_regs.h instead of hardcoded hex addresses.
00265  * Keeps PFS writes raw because the audited rx_mpc helpers stop
00266  * short of writing PFS for MTCLK/TCLK pins (HAL bug, see memory
00267  * project_motor_pwm_hal_bugs.md). */
00268 static bool encoders_init(void)
00269 {
00270     /* Step 1: enable MTU (MSTPA9) + TPU (MSTPA13) module clocks. */
00271     *prcr_reg() = k_rx_prcr_unlock_all;
00272     system_regs()->mstpca &= ~(uint32_t)((1UL << 9) | (uint32_t)k_tpu_mstpca_mstp13);
00273     *prcr_reg() = k_rx_prcr_lock;
00274
00275     /* Step 2 + 3: RX72N MPC pin setup requires the sequence PMR=0, write PFS,
00276      * PMR=1, MTCLKA/MTCLKB on P24/P25 take PSEL=0x02 (the production header's
00277      * old k_psel_mtu_phase=0x03 was WRONG -- 0x02 is the MTCLK PSEL on RX72N).
00278      * For TPU TCLK keep our verified per-pin values. */
00279     volatile uint8_t *pwpr = (volatile uint8_t *)0x0008C11FU;
00280     volatile uint8_t *p2pmr = (volatile uint8_t *)0x0008C062U;
00281     volatile uint8_t *papmr = (volatile uint8_t *)0x0008C06AU;
00282     volatile uint8_t *pbpmr = (volatile uint8_t *)0x0008C06BU;
00283     volatile uint8_t *pcpmr = (volatile uint8_t *)0x0008C06CU;
00284
00285     /* clear PMR for all encoder input pins so PFS writes take effect */
00286     *p2pmr &= (uint8_t)~(uint8_t)((1U << 4) | (1U << 5));
00287     *papmr &= (uint8_t)~(uint8_t)((1U << 1) | (1U << 3));
00288     *pcpmr &= (uint8_t)~(uint8_t)((1U << 0) | (1U << 2) | (1U << 5));
00289     *pbpmr &= (uint8_t)~(uint8_t)(1U << 3);
00290
00291     /* PFS writes */
00292     *pwpr = 0x00U; *pwpr = 0x40U;
00293     REG8_AT(0x0008C140U + 2U * 8U + 4U) = 0x02U; /* P24 MTCLKA -- enc0 A (MTU PSEL=0x02, RX72N HW
manual R01UH0824EJ0111 Table 23.6) */
00294     REG8_AT(0x0008C140U + 2U * 8U + 5U) = 0x02U; /* P25 MTCLKB -- enc0 B */
00295     REG8_AT(0x0008C140U + 10U * 8U + 1U) = 0x02U; /* PA1 MTCLKC -- enc1 A */
00296     REG8_AT(0x0008C140U + 12U * 8U + 5U) = 0x02U; /* PC5 MTCLKD -- enc1 B */
00297     REG8_AT(0x0008C140U + 12U * 8U + 2U) = 0x03U; /* PC2 TCLKA -- enc3 A (TPU1, reads BL per harness) */
00298     REG8_AT(0x0008C140U + 10U * 8U + 3U) = 0x04U; /* PA3 TCLKB -- enc3 B (TPU1, reads BL per harness) */
00299     REG8_AT(0x0008C140U + 12U * 8U + 0U) = 0x03U; /* PC0 TCLKC -- enc2 A (TPU2, reads BR per harness) */
00300     REG8_AT(0x0008C140U + 11U * 8U + 3U) = 0x04U; /* PB3 TCLKD -- enc2 B (TPU2, reads BR per harness) */
00301     *pwpr = 0x00U; *pwpr = 0x80U;
00302
00303     /* set PMR=1 to route the pad to the peripheral */
00304     *p2pmr |= (uint8_t)((1U << 4) | (1U << 5));
00305     *papmr |= (uint8_t)((1U << 1) | (1U << 3));
00306     *pcpmr |= (uint8_t)((1U << 0) | (1U << 2) | (1U << 5));
00307     *pbpmr |= (uint8_t)(1U << 3);
00308
00309     /* Step 4: phase counting mode 1 on MTU1 + MTU2 via accessors. */
00310     volatile rx_mtu_channel_regs_t *m1 = mtu1();
00311     m1->tcrcr = 0;
00312     m1->tmdr = 0x04U; /* phase counting mode 1 */
00313     m1->tiorh = 0;
00314     m1->tiorl = 0;
00315     m1->tier = 0;
00316     m1->tsr = 0;
00317     m1->tcnt = 0;
00318     volatile rx_mtu_channel_regs_t *m2 = mtu2();
00319     m2->tcrcr = 0;
00320     m2->tmdr = 0x04U;
00321     m2->tiorh = 0;
00322     m2->tiorl = 0;
00323     m2->tier = 0;

```

```

00324 m2->tsr = 0;
00325 m2->tcnt = 0;
00326
00327 /* Step 5: configure TPU1 + TPU2 for phase counting mode 1 via accessors. */
00328 volatile rx_tpu_regs_t* t1 = tpu1();
00329 t1->tcr = 0;
00330 t1->tmdr = 0x04U;
00331 t1->tcnt = 0;
00332 volatile rx_tpu_regs_t* t2 = tpu2();
00333 t2->tcr = 0;
00334 t2->tmdr = 0x04U;
00335 t2->tcnt = 0;
00336
00337 /* Step 6: disable noise filter on TPU1/TPU2 explicitly (HW reset clears,
00338  * but ensure leftover state from a prior run is gone). */
00339 tpu_control()->nfcr[1] = 0;
00340 tpu_control()->nfcr[2] = 0;
00341
00342 /* Step 7: start counters. */
00343 mtu_tstra()->tstr |= (uint8_t)(k_mtu_tstr_cst1 | k_mtu_tstr_cst2);
00344 tpu_control()->tstr |= (uint8_t)(k_tpu_tstr_cst1 | k_tpu_tstr_cst2);
00345
00346 return true;
00347 }
00348
00349 static bool encoder_read_raw(uint8_t motor_idx, uint16_t *out_count)
00350 {
00351     /* Direct register read -- bypass the lib (rx_encoder_read_raw works
00352      * but for symmetry with the bypass init). */
00353     static const uintptr_t k_tcnt_addr[k_motor_count] = {
00354         0x000C1386U, /* MTU1 */
00355         0x000C1406U, /* MTU2 */
00356         0x00088120U + 6U, /* TPU1 */
00357         0x00088130U + 6U, /* TPU2 */
00358     };
00359     *out_count = REG16_AT(k_tcnt_addr[motor_idx]);
00360     return true;
00361 }
00362
00363 /*
=====
00364 * UART helpers -- print 4 motor counts on one line.
00365 *
=====
*/
00366 static void print_counts(const char *label, uint16_t c[k_motor_count])
00367 {
00368     sci9_debug_puts(label);
00369     sci9_debug_puts(" m0=");
00370     sci9_debug_puthex16(c[0]);
00371     sci9_debug_puts(" m1=");
00372     sci9_debug_puthex16(c[1]);
00373     sci9_debug_puts(" m2=");
00374     sci9_debug_puthex16(c[2]);
00375     sci9_debug_puts(" m3=");
00376     sci9_debug_puthex16(c[3]);
00377     sci9_debug_puts("\n");
00378 }
00379
00380 static void error_park(const char *msg)
00381 {
00382     sci9_debug_puts("FATAL ");
00383     sci9_debug_puts(msg);
00384     sci9_debug_puts("\n");
00385     led(k_port_b, k_bit_led5, true);
00386     for (;;) { __asm__ volatile("nop"); }
00387 }
00388
00389 /*
=====
00390 * Test sequence
00391 *
=====
*/
00392 typedef enum : uint32_t {
00393     k_spin_ms = 3000U, /* drive motors this long */
00394     k_settle_ms = 500U, /* let everything stop */
00395     k_min_delta = 50U, /* minimum ticks expected per encoder over k_spin_ms */
00396 } test_timing_t;
00397
00398 int main(void)
00399 {
00400     clock_init();
00401     leds_init();
00402     led(k_port_b, k_bit_led1, true);
00403
00404     sci9_debug_init();

```

```

00405 motor_drv_gpio_init();
00406
00407 static rx_motor_handle_t s_motors[k_motor_count];
00408 if (!motor_pwm_init(s_motors)) { error_park("motor_pwm_init"); }
00409 if (!encoders_init()) { error_park("encoders_init"); }
00410
00411 /* === Sanity check: can MTU1 count AT ALL using its internal clock?
00412 * This bypasses the encoder pin routing entirely. Set TCR.TPSC=000
00413 * (PCLK/1) and TMDR=0 (normal mode). At 60 MHz PCLKB the counter should
00414 * wrap nearly continuously. If TCNT stays at 0x0000 here too, MTU1
00415 * itself isn't running -- not a pin routing issue. */
00416 REG8_AT(0x000C1280U) &= (uint8_t)~(uint8_t)((1U < 1) | (1U < 2)); /* stop MTU1, MTU2 */
00417 REG8_AT(0x000C1380U) = 0x00U; /* TCR = TPSC=000 (PCLK/1), CKEG=0, CCLR=0 */
00418 REG8_AT(0x000C1381U) = 0x00U; /* TMDR = normal mode */
00419 REG16_AT(0x000C1386U) = 0; /* TCNT = 0 */
00420 REG8_AT(0x000C1280U) |= (uint8_t)(1U < 1); /* start MTU1 */
00421 busy_wait_ms(10);
00422 uint16_t internal_cnt = REG16_AT(0x000C1386U);
00423 sci9_debug_puts("MTU1 internal-clock test (10 ms PCLK/1): TCNT=0x");
00424 sci9_debug_puthex16(internal_cnt);
00425 sci9_debug_puts(internal_cnt != 0 ? " ALIVE\n" : " DEAD\n");
00426 /* Restore MTU1+MTU2 to phase counting mode for the spin loop. */
00427 REG8_AT(0x000C1280U) &= (uint8_t)~(uint8_t)((1U < 1) | (1U < 2));
00428 REG8_AT(0x000C1380U) = 0x00U;
00429 REG8_AT(0x000C1381U) = 0x04U;
00430 REG16_AT(0x000C1386U) = 0;
00431 REG8_AT(0x000C1400U) = 0x00U;
00432 REG8_AT(0x000C1401U) = 0x04U;
00433 REG16_AT(0x000C1406U) = 0;
00434 REG8_AT(0x000C1280U) |= (uint8_t)((1U < 1) | (1U < 2));
00435
00436 /* Loop the test forever so any capture window catches a full cycle.
00437 * Each iteration: print banner, sample, spin 3s, sample, stop, summarize,
00438 * idle 2s, repeat. */
00439 uint16_t iter = 0;
00440 for (;;) {
00441     sci9_debug_puts("\n=== ENCODER_TEST iter=");
00442     sci9_debug_puthex16(iter);
00443     sci9_debug_puts(" ===\n");
00444
00445     uint16_t before[k_motor_count] = {0};
00446     for (uint8_t i = 0; i < k_motor_count; i++) {
00447         if (!encoder_read_raw(i, &before[i])) { error_park("encoder_read_raw before"); }
00448     }
00449     print_counts("before", before);
00450
00451     /* Dump MTU1 + TSTRA + a few port registers using production accessors. */
00452     {
00453         volatile rx_mtu_channel_regs_t* m = mtu1();
00454         sci9_debug_puts("regs: TCR="); sci9_debug_puthex16(m->tcrc);
00455         sci9_debug_puts(" TMDR="); sci9_debug_puthex16(m->tmdr);
00456         sci9_debug_puts(" TIORH="); sci9_debug_puthex16(m->tiorh);
00457         sci9_debug_puts(" TIER="); sci9_debug_puthex16(m->tier);
00458         sci9_debug_puts(" TSR="); sci9_debug_puthex16(m->tsr);
00459         sci9_debug_puts(" TCNT="); sci9_debug_puthex16(m->tcnt);
00460         sci9_debug_puts(" TSTRA="); sci9_debug_puthex16(mtu_tstra()->tstr);
00461         sci9_debug_puts(" P24PFS="); sci9_debug_puthex16((volatile uint8_t*)0x0008C154U);
00462         sci9_debug_puts(" P25PFS="); sci9_debug_puthex16((volatile uint8_t*)0x0008C155U);
00463         sci9_debug_puts(" P2PMR="); sci9_debug_puthex16((volatile uint8_t*)0x0008C062U);
00464         sci9_debug_puts(" P2PIDR="); sci9_debug_puthex16((volatile uint8_t*)0x0008C042U);
00465         sci9_debug_puts("\n");
00466     }
00467     sci9_debug_puts("spinning motors (per-motor sign applied) for 3s, sampling each second\n");
00468     led(k_port_7, k_bit_led2, true);
00469     for (uint8_t i = 0; i < k_motor_count; i++) {
00470         const float duty = 50.0F * (float)k_motors[i].direction_sign;
00471         (void)rx_motor_set_duty(&s_motors[i], duty);
00472     }
00473
00474     /* Burst-read MTU1.TCNT in a tight loop right after zeroing it -- if
00475     * MTU is counting fast (TSR.TCFV is set in our dump => overflow has
00476     * fired at least once), the values will move quickly across this loop,
00477     * even though the periodic 1-second sample lands at 0. */
00478     mtu1()->tcnt = 0;
00479     sci9_debug_puts("burst MTU1.TCNT:");
00480     for (uint8_t s = 0; s < 8; s++) {
00481         busy_wait_ms(1);
00482         uint16_t c = mtu1()->tcnt;
00483         sci9_debug_puts(" ");
00484         sci9_debug_puthex16(c);
00485     }
00486     sci9_debug_puts("\n");
00487
00488     /* Sample 3 times during the spin so we can see if counters move at all. */
00489     volatile uint8_t* p2_pidr = (volatile uint8_t*)0x0008C042U;
00490     volatile uint8_t* pa_pidr = (volatile uint8_t*)0x0008C044U;
00491     volatile uint8_t* pc_pidr = (volatile uint8_t*)0x0008C04CU;

```

```

00492 uint16_t mid[k_motor_count] = {0};
00493 for (uint8_t s = 0; s < 3; s++) {
00494     busy_wait_ms(k_spin_ms / 3U);
00495     for (uint8_t i = 0; i < k_motor_count; i++) {
00496         (void)encoder_read_raw(i, &mid[i]);
00497     }
00498     sci9_debug_puts("mid ");
00499     sci9_debug_puthex16(mid[0]); sci9_debug_puts(" ");
00500     sci9_debug_puthex16(mid[1]); sci9_debug_puts(" ");
00501     sci9_debug_puthex16(mid[2]); sci9_debug_puts(" ");
00502     sci9_debug_puthex16(mid[3]);
00503     sci9_debug_puts(" pidr2="); sci9_debug_puthex16(*p2_pidr);
00504     sci9_debug_puts(" pidrA="); sci9_debug_puthex16(*pa_pidr);
00505     sci9_debug_puts(" pidrC="); sci9_debug_puthex16(*pc_pidr);
00506     sci9_debug_puts("\n");
00507 }
00508
00509 uint16_t after[k_motor_count] = {0};
00510 for (uint8_t i = 0; i < k_motor_count; i++) {
00511     if (!encoder_read_raw(i, &after[i])) { error_park("encoder_read_raw after"); }
00512 }
00513 print_counts("after ", after);
00514
00515 for (uint8_t i = 0; i < k_motor_count; i++) {
00516     (void)rx_motor_set_duty(&s_motors[i], 0.0F);
00517 }
00518 led(k_port_7, k_bit_led2, false);
00519 sci9_debug_puts("motors stopped\n");
00520 busy_wait_ms(k_settle_ms);
00521
00522 uint16_t delta[k_motor_count];
00523 for (uint8_t i = 0; i < k_motor_count; i++) {
00524     delta[i] = (uint16_t)(after[i] - before[i]);
00525 }
00526 print_counts("delta ", delta);
00527
00528 bool pass = true;
00529 for (uint8_t i = 0; i < k_motor_count; i++) {
00530     int32_t signed_delta = (int32_t)(int16_t)delta[i];
00531     uint32_t mag = (uint32_t)((signed_delta < 0) ? -signed_delta : signed_delta);
00532     if (mag < k_min_delta) {
00533         pass = false;
00534         sci9_debug_puts("encoder FAIL idx=");
00535         sci9_debug_puthex16((uint16_t)i);
00536         sci9_debug_puts("\n");
00537     }
00538 }
00539 sci9_debug_puts(pass ? "summary PASS\n" : "summary FAIL\n");
00540 if (pass) { led(k_port_b, k_bit_led4, true); }
00541 else { led(k_port_b, k_bit_led5, true); }
00542
00543 /* Idle 2s then repeat. */
00544 busy_wait_ms(2000);
00545 iter++;
00546 }
00547 }

```

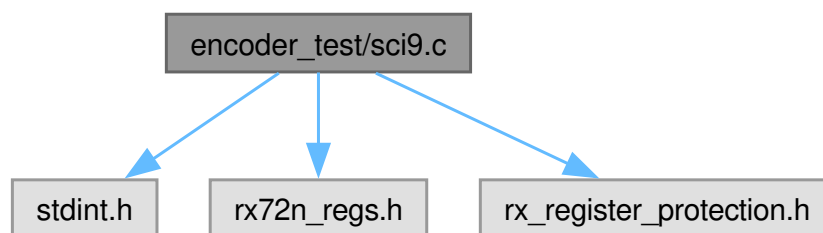
4.7 encoder_test/sci9.c File Reference

```

#include <stdint.h>
#include "rx72n_regs.h"
#include "rx_register_protection.h"

```

Include dependency graph for sci9.c:



Enumerations

- enum [sci9_addrs_t](#) : uintptr_t { [k_sci9_base](#) = 0x000D0020U }
- enum [sci9_cfg_t](#) : uint8_t {
[k_psel_sci9](#) = 0x0AU , [k_pb6_mask](#) = (uint8_t)(1U << 6U) , [k_pb7_mask](#) = (uint8_t)(1U << 7U) , [k_mstpc_sci9_bit](#) = 26U ,
[k_pwpr_unlock_b0](#) = 0x00U , [k_pwpr_pfswe](#) = 0x40U , [k_pwpr_b0wi](#) = 0x80U , [k_brr_115200](#) = 31U ,
[k_smr_8n1_async](#) = 0x00U , [k_scr_te_only](#) = 0x20U , [k_scr_txrx_enabled](#) = 0x30U ,
[k_scr_disabled](#) = 0x00U ,
[k_ssr_tdre](#) = 0x80U , [k_scmr_default](#) = 0xF2U , [k_semr_default](#) = 0x00U , [k_hex_nibble_mask](#) = 0x0FU }
- enum [sci9_settle_t](#) : uint16_t { [k_brr_settle_loops](#) = 1000U }
- enum [hex_shifts_t](#) : uint8_t { [k_hex_shift_init32](#) = 28U , [k_hex_shift_init16](#) = 12U ,
[k_hex_shift_step](#) = 4U }

Functions

- static volatile uint8_t * [sci9_smr](#) (void)
- static volatile uint8_t * [sci9_brr](#) (void)
- static volatile uint8_t * [sci9_scr](#) (void)
- static volatile uint8_t * [sci9_tdr](#) (void)
- static volatile uint8_t * [sci9_ssr](#) (void)
- static volatile uint8_t * [sci9_scmr](#) (void)
- static volatile uint8_t * [sci9_semr](#) (void)
- void [sci9_debug_init](#) (void)
- void [sci9_debug_putc](#) (char c)
- void [sci9_debug_puts](#) (const char *s)
- void [sci9_debug_puthex32](#) (uint32_t v)
- void [sci9_debug_puthex16](#) (uint16_t v)

4.7.1 Enumeration Type Documentation

4.7.1.1 hex_shifts_t

enum [hex_shifts_t](#) : uint8_t

Enumerator

k_hex_shift_init32	
k_hex_shift_init16	
k_hex_shift_step	

Definition at line 62 of file [sci9.c](#).

```

00062      : uint8_t {
00063      k_hex_shift_init32 = 28U,
00064      k_hex_shift_init16 = 12U,
00065      k_hex_shift_step   = 4U,
00066 } hex_shifts_t;
```

4.7.1.2 sci9_addrs_t

```
enum sci9_addrs_t : uintptr_t
```

Enumerator

k_sci9_base	
-------------	--

Definition at line 35 of file [sci9.c](#).

```
00035     : uintptr_t {
00036     k_sci9_base = 0x000D0020U,
00037 } sci9_addrs_t;
```

4.7.1.3 sci9_cfg_t

```
enum sci9_cfg_t : uint8_t
```

Enumerator

k_psel_sci9	SCI9 RXD/TXD function
k_pb6_mask	
k_pb7_mask	
k_mstpc_sci9_bit	
k_pwpr_unlock_b0	
k_pwpr_pfswe	
k_pwpr_b0wi	
k_brr_115200	
k_smr_8n1_async	
k_scr_te_only	CKE=00, MPIE=0, RIE=0, TE=1, RE=0
k_scr_txrx_enabled	CKE=00, MPIE=0, RIE=0, TE=1, RE=1
k_scr_disabled	
k_ssr_tdre	
k_scmr_default	
k_semr_default	
k_hex_nibble_mask	

Definition at line 39 of file [sci9.c](#).

```
00039     : uint8_t {
00040     k_psel_sci9      = 0x0AU, /**< SCI9 RXD/TXD function */
00041     k_pb6_mask       = (uint8_t)(1U « 6U),
00042     k_pb7_mask       = (uint8_t)(1U « 7U),
00043     k_mstpc_sci9_bit = 26U, /** MSTPCRC.MSTPC26 per RX72N HW manual page 410 */
00044     k_pwpr_unlock_b0 = 0x00U,
00045     k_pwpr_pfswe     = 0x40U,
00046     k_pwpr_b0wi      = 0x80U,
00047     k_brr_115200     = 31U, /** SCI9 uses PCLKA=120 MHz (SCI7-11 group); BRR=31 @ CKS=0 ABCS=0 -> ~117
                                kbaud (1.7% over 115200) */
00048     k_smr_8n1_async  = 0x00U,
00049     k_scr_te_only    = 0x20U, /**< CKE=00, MPIE=0, RIE=0, TE=1, RE=0 */
00050     k_scr_txrx_enabled = 0x30U, /**< CKE=00, MPIE=0, RIE=0, TE=1, RE=1 */
00051     k_scr_disabled   = 0x00U,
00052     k_ssr_tdre       = 0x80U,
00053     k_scmr_default   = 0xF2U,
00054     k_semr_default   = 0x00U,
00055     k_hex_nibble_mask = 0x0FU,
00056 } sci9_cfg_t;
```

4.7.1.4 sci9_settle_t

enum [sci9_settle_t](#) : uint16_t

Enumerator

k_brr_settle_loops	Match production uart.c (~8.68 us @ 115200)
--------------------	---

Definition at line 58 of file [sci9.c](#).

```
00058      : uint16_t {
00059      k_brr_settle_loops = 1000U, /**< Match production uart.c (~8.68 us @ 115200) */
00060 } sci9_settle_t;
```

4.7.2 Function Documentation

4.7.2.1 sci9_brr()

volatile uint8_t * sci9_brr (
void) [inline], [static]

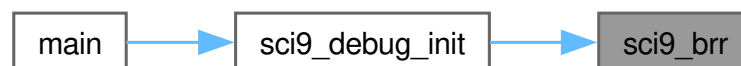
Definition at line 76 of file [sci9.c](#).

```
00077 {
00078      return (volatile uint8_t *) (k_sci9_base + 1U);
00079 }
```

References [k_sci9_base](#).

Referenced by [sci9_debug_init\(\)](#).

Here is the caller graph for this function:



4.7.2.2 sci9_debug_init()

void sci9_debug_init (
void)

Definition at line 104 of file [sci9.c](#).

```
00105 {
00106      /* Step 1: release SCI9 from module stop. SCI9 = MSTPCRC bit 26 per
00107      * RX72N HW manual page 410 (NOT MSTPCRB.bit22 -- that's PDC). */
00108      *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00109      system_regs()->mstpcrc &= ~(uint32_t)(1UL « (uint32_t)k_mstpc_sci9_bit);
00110      *prcr_reg() = k_rx_prcr_lock;
00111
00112      /* Step 2: configure PB6 (RXD9) and PB7 (TXD9) via MPC. Use the struct
00113      * accessor so the compiler computes the correct PFS offsets via
00114      * static_assert-checked layout. */
00115      mpc()->pwpr = k_pwpr_unlock_b0; /* clear B0WI */
00116      mpc()->pwpr = k_pwpr_pfswe; /* allow PFS writes */
00117      mpc()->pb6pfs = k_psel_sci9;
00118      mpc()->pb7pfs = k_psel_sci9;
00119      mpc()->pwpr = k_pwpr_unlock_b0; /* clear PFSWE */
00120      mpc()->pwpr = k_pwpr_b0wi; /* re-protect */
00121 }
```

```

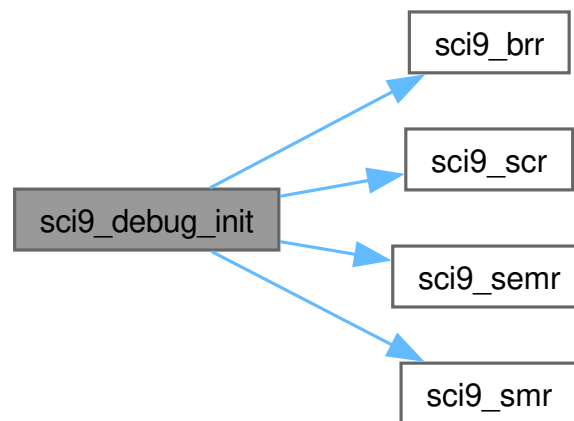
00122  /* Step 3: PDR before PMR (production uart.c does this). PB7 = output
00123  * for TX, PB6 = input for RX. Then flip PMR to peripheral mode. */
00124  portb()->pdr |= (uint8_t)k_pb7_mask;
00125  portb()->pdr &= (uint8_t)~k_pb6_mask;
00126  portb()->pmr |= (uint8_t)(k_pb6_mask | k_pb7_mask);
00127
00128  /* Step 4: SCI9 init. Match the production order:
00129  *   SCR=0 -> SMR -> BRR -> settle -> SCR=TE|RE -> SEMR.
00130  *   Don't touch SCMR (reset default 0xF2 is correct). */
00131  *sci9_scr() = k_scr_disabled;
00132  *sci9_smr() = k_smr_8n1_async;
00133  *sci9_brr() = k_brr_115200;
00134
00135  /* Wait roughly one bit time before enabling TX/RX. */
00136  for (volatile uint32_t i = 0U; i < (uint32_t)k_brr_settle_loops; i++) {
00137      __asm__ volatile("nop");
00138  }
00139
00140  *sci9_scr() = k_scr_txxr_enabled;
00141  *sci9_semr() = k_semr_default;
00142  }

```

References [k_brr_115200](#), [k_brr_settle_loops](#), [k_mstpc_sci9_bit](#), [k_pb6_mask](#), [k_pb7_mask](#), [k_psel_sci9](#), [k_pwpr_b0wi](#), [k_pwpr_pfswe](#), [k_pwpr_unlock_b0](#), [k_scr_disabled](#), [k_scr_txxr_enabled](#), [k_semr_default](#), [k_smr_8n1_async](#), [sci9_brr\(\)](#), [sci9_scr\(\)](#), [sci9_semr\(\)](#), and [sci9_smr\(\)](#).

Referenced by [main\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.7.2.3 sci9_debug_putc()

```

void sci9_debug_putc (
    char c)

```

Definition at line 147 of file [sci9.c](#).

```

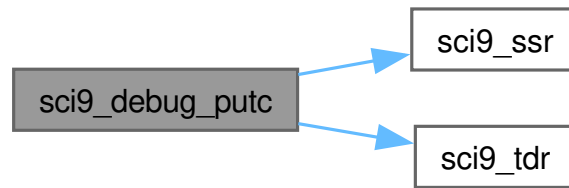
00148 {
00149     while ((*sci9_ssr() & (uint8_t)k_ssr_tdre) == 0U) {
00150         /* spin */
00151     }
00152     *sci9_tdr() = (uint8_t)c;
00153 }

```

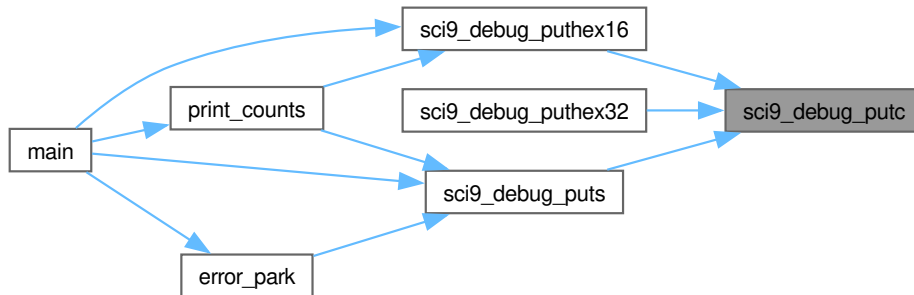
References [k_ssr_tdre](#), [sci9_ssr\(\)](#), and [sci9_tdr\(\)](#).

Referenced by [sci9_debug_puthex16\(\)](#), [sci9_debug_puthex32\(\)](#), and [sci9_debug_puts\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.7.2.4 sci9_debug_puthex16()

```
void sci9_debug_puthex16 (
    uint16_t v)
```

Definition at line 179 of file [sci9.c](#).

```
00180 {
00181     static const char hex_digits[] = "0123456789ABCDEF";
00182     sci9_debug_putc('0');
00183     sci9_debug_putc('x');
00184     for (int8_t shift = (int8_t)k_hex_shift_init16; shift >= 0;
00185         shift -= (int8_t)k_hex_shift_step) {
00186         sci9_debug_putc(hex_digits[(v » (uint8_t)shift) & (uint8_t)k_hex_nibble_mask]);
00187     }
00188 }
```

References [k_hex_nibble_mask](#), [k_hex_shift_init16](#), [k_hex_shift_step](#), and [sci9_debug_putc\(\)](#).

Referenced by [main\(\)](#), and [print_counts\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.7.2.5 sci9_debug_puthex32()

```
void sci9_debug_puthex32 (
    uint32_t v)
```

Definition at line 168 of file [sci9.c](#).

```
00169 {
00170     static const char hex_digits[] = "0123456789ABCDEF";
00171     sci9_debug_putc('0');
00172     sci9_debug_putc('x');
00173     for (int8_t shift = (int8_t)k_hex_shift_init32; shift >= 0;
00174          shift -= (int8_t)k_hex_shift_step) {
00175         sci9_debug_putc(hex_digits[(v » (uint8_t)shift) & (uint8_t)k_hex_nibble_mask]);
00176     }
00177 }
```

References [k_hex_nibble_mask](#), [k_hex_shift_init32](#), [k_hex_shift_step](#), and [sci9_debug_putc\(\)](#).

Here is the call graph for this function:



4.7.2.6 sci9_debug_puts()

```
void sci9_debug_puts (
    const char * s)
```

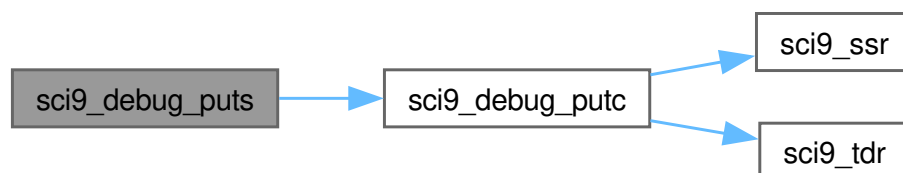
Definition at line 155 of file [sci9.c](#).

```
00156 {
00157     if (s == (const char *)0) {
00158         return;
00159     }
00160     while (*s != '\0') {
00161         if (*s == '\n') {
00162             sci9_debug_putc('\r');
00163         }
00164         sci9_debug_putc(*s++);
00165     }
00166 }
```

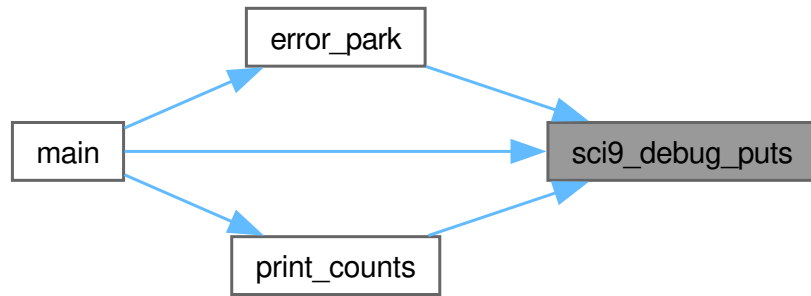
References [sci9_debug_putc\(\)](#).

Referenced by [error_park\(\)](#), [main\(\)](#), and [print_counts\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.7.2.7 sci9_scmr()

```
volatile uint8_t * sci9_scmr (
    void ) [inline], [static]
```

Definition at line 92 of file [sci9.c](#).

```
00093 {
00094     return (volatile uint8_t *) (k_sci9_base + 6U);
00095 }
```

References [k_sci9_base](#).

4.7.2.8 sci9_scr()

```
volatile uint8_t * sci9_scr (
    void ) [inline], [static]
```

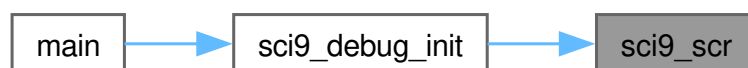
Definition at line 80 of file [sci9.c](#).

```
00081 {
00082     return (volatile uint8_t *) (k_sci9_base + 2U);
00083 }
```

References [k_sci9_base](#).

Referenced by [sci9_debug_init\(\)](#).

Here is the caller graph for this function:



4.7.2.9 sci9_semr()

```
volatile uint8_t * sci9_semr (
    void ) [inline], [static]
```

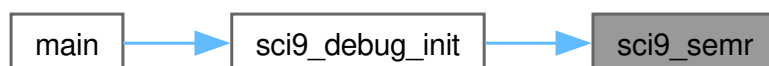
Definition at line 96 of file [sci9.c](#).

```
00097 {
00098     return (volatile uint8_t *) (k_sci9_base + 7U);
00099 }
```

References [k_sci9_base](#).

Referenced by [sci9_debug_init\(\)](#).

Here is the caller graph for this function:



4.7.2.10 sci9_smr()

```
volatile uint8_t * sci9_smr (
    void ) [inline], [static]
```

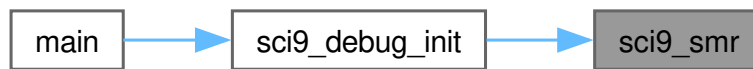
Definition at line 72 of file [sci9.c](#).

```
00073 {
00074     return (volatile uint8_t *) (k_sci9_base + 0U);
00075 }
```

References [k_sci9_base](#).

Referenced by [sci9_debug_init\(\)](#).

Here is the caller graph for this function:



4.7.2.11 sci9_ssr()

```
volatile uint8_t * sci9_ssr (
    void ) [inline], [static]
```

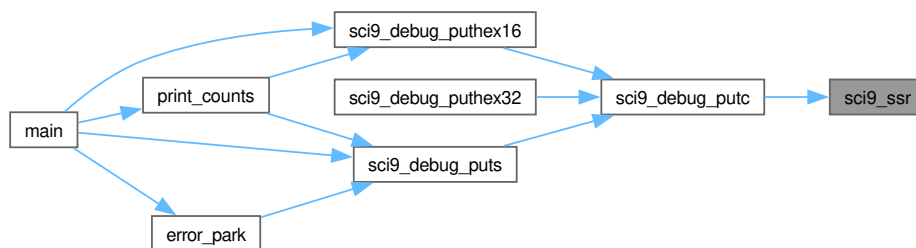
Definition at line 88 of file [sci9.c](#).

```
00089 {
00090     return (volatile uint8_t *) (k_sci9_base + 4U);
00091 }
```

References [k_sci9_base](#).

Referenced by [sci9_debug_putc\(\)](#).

Here is the caller graph for this function:



4.7.2.12 sci9_tdr()

```
volatile uint8_t * sci9_tdr (
    void ) [inline], [static]
```

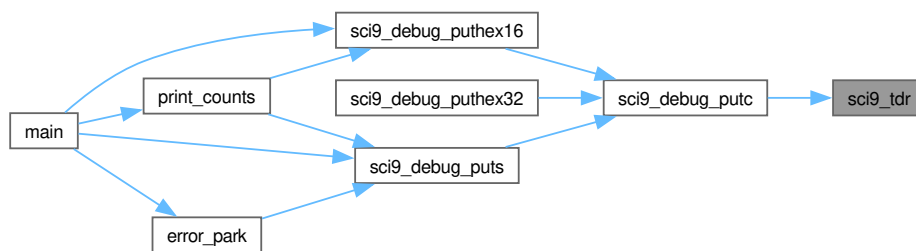
Definition at line 84 of file [sci9.c](#).

```
00085 {
00086     return (volatile uint8_t *) (k_sci9_base + 3U);
00087 }
```

References [k_sci9_base](#).

Referenced by [sci9_debug_putc\(\)](#).

Here is the caller graph for this function:



4.8 sci9.c

[Go to the documentation of this file.](#)

```
00001 /**
00002  * @file sci9_debug.c
00003  * @brief Minimal polled-mode SCI9 (TXD9 = PB7, RXD9 = PB6) debug UART for encoder_test.
00004  *
00005  * @details
00006  * The STAR board routes SCI9 to the on-board CY7C65213 USB-to-UART bridge
00007  * which appears as /dev/ttyACM0 on the host (or COMx on Windows). This
00008  * file provides just enough init + putc/puts to emit ASCII status from
00009  * the firmware so we can debug the USB CDC bring-up without needing the
00010  * USB device to actually enumerate.
00011  *
00012  * Uses the rx_hal struct accessors (mpc(), portb(), system_regs(), prcr_reg())
00013  * so the addresses are all compiler-verified via static_assert.
00014  *
00015  * Baud = 115200. SCI9 is in the SCI7-11 group that uses PCLKA (120 MHz),
00016  * not PCLKB. At CKS=0, ABCS=0:
00017  *   BRR = (PCLKA / (32 * baud)) - 1 = 120000000 / 3686400 - 1 = 31.55 -> 31
00018  *   Actual baud = PCLKA / (32 * (BRR+1)) = 117187 Hz (+1.7% vs 115200, in tolerance).
00019  *
00020  * SPDX-License-Identifier: MIT
00021  * @copyright Copyright (c) 2026 Locked Inc.
00022  */
00023
00024 #include <stdint.h>
00025
00026 #include "rx72n_regs.h"
00027 #include "rx_register_protection.h"
00028
00029 /**
00030  * SCI9 register access via raw addresses (rx72n_sci_regs.h doesn't expose
00031  * the per-channel struct as cleanly as we'd like for a quick polled driver).
00032  * The base 0x000D0020 is taken from the verified header.
00033  * Register offsets per RX72N HW manual: SMR=0, BRR=1, SCR=2, TDR=3, SSR=4.
00034  */
00035 typedef enum : uintptr_t {
00036     k_sci9_base = 0x000D0020U,
00037 } sci9_addrs_t;
00038
```

```

00039 typedef enum : uint8_t {
00040     k_psel_sci9      = 0x0AU, /**< SCI9 RXD/TXD function */
00041     k_pb6_mask       = (uint8_t)(1U « 6U),
00042     k_pb7_mask       = (uint8_t)(1U « 7U),
00043     k_mstpc_sci9_bit = 26U, /** MSTPCRC.MSTPC26 per RX72N HW manual page 410 */
00044     k_pwpr_unlock_b0 = 0x00U,
00045     k_pwpr_pfswe     = 0x40U,
00046     k_pwpr_b0wi      = 0x80U,
00047     k_brr_115200     = 31U, /** SCI9 uses PCLKA=120 MHz (SCI7-11 group); BRR=31 @ CKS=0 ABCS=0 -> ~117
kbaud (1.7% over 115200) */
00048     k_smr_8n1_async  = 0x00U,
00049     k_scr_te_only    = 0x20U, /**< CKE=00, MPIE=0, RIE=0, TE=1, RE=0 */
00050     k_scr_txrx_enabled = 0x30U, /**< CKE=00, MPIE=0, RIE=0, TE=1, RE=1 */
00051     k_scr_disabled    = 0x00U,
00052     k_ssr_tdre        = 0x80U,
00053     k_scmr_default    = 0xF2U,
00054     k_semr_default    = 0x00U,
00055     k_hex_nibble_mask = 0x0FU,
00056 } sci9_cfg_t;
00057
00058 typedef enum : uint16_t {
00059     k_brr_settle_loops = 1000U, /**< Match production uart.c (~8.68 us @ 115200) */
00060 } sci9_settle_t;
00061
00062 typedef enum : uint8_t {
00063     k_hex_shift_init32 = 28U,
00064     k_hex_shift_init16 = 12U,
00065     k_hex_shift_step   = 4U,
00066 } hex_shifts_t;
00067
00068 /**
=====
00069 * SCI9 register accessors (raw addresses since the header doesn't give us a
00070 * per-channel struct for the SCIi/SCIj quirks).
00071 *
=====
*/
00072 static inline volatile uint8_t *sci9_smr(void)
00073 {
00074     return (volatile uint8_t *) (k_sci9_base + 0U);
00075 }
00076 static inline volatile uint8_t *sci9_brr(void)
00077 {
00078     return (volatile uint8_t *) (k_sci9_base + 1U);
00079 }
00080 static inline volatile uint8_t *sci9_scr(void)
00081 {
00082     return (volatile uint8_t *) (k_sci9_base + 2U);
00083 }
00084 static inline volatile uint8_t *sci9_tdr(void)
00085 {
00086     return (volatile uint8_t *) (k_sci9_base + 3U);
00087 }
00088 static inline volatile uint8_t *sci9_ssr(void)
00089 {
00090     return (volatile uint8_t *) (k_sci9_base + 4U);
00091 }
00092 static inline volatile uint8_t *sci9_scmr(void)
00093 {
00094     return (volatile uint8_t *) (k_sci9_base + 6U);
00095 }
00096 static inline volatile uint8_t *sci9_semr(void)
00097 {
00098     return (volatile uint8_t *) (k_sci9_base + 7U);
00099 }
00100
00101 /**
=====
00102 * Initialise SCI9 for 115200 8N1 polled output on PB7 (TXD9).
00103 *
=====
*/
00104 void sci9_debug_init(void)
00105 {
00106     /** Step 1: release SCI9 from module stop. SCI9 = MSTPCRC bit 26 per
00107     * RX72N HW manual page 410 (NOT MSTPCRB.bit22 -- that's PDC). */
00108     *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00109     system_regs()->mstpcrc &= ~(uint32_t)(1UL « (uint32_t)k_mstpc_sci9_bit);
00110     *prcr_reg() = k_rx_prcr_lock;
00111
00112     /** Step 2: configure PB6 (RXD9) and PB7 (TXD9) via MPC. Use the struct
00113     * accessor so the compiler computes the correct PFS offsets via
00114     * static_assert-checked layout. */
00115     mpc()->pwpr = k_pwpr_unlock_b0; /** clear B0WI */
00116     mpc()->pwpr = k_pwpr_pfswe; /** allow PFS writes */
00117     mpc()->pb6pfs = k_psel_sci9;
00118     mpc()->pb7pfs = k_psel_sci9;

```

```

00119 mpc()->pwpr = k_pwpr_unlock_b0; /* clear PFSWE */
00120 mpc()->pwpr = k_pwpr_b0wi; /* re-protect */
00121
00122 /* Step 3: PDR before PMR (production uart.c does this). PB7 = output
00123  * for TX, PB6 = input for RX. Then flip PMR to peripheral mode. */
00124 portb()->pdr |= (uint8_t)k_pb7_mask;
00125 portb()->pdr &= (uint8_t)~k_pb6_mask;
00126 portb()->pmr |= (uint8_t)(k_pb6_mask | k_pb7_mask);
00127
00128 /* Step 4: SCI9 init. Match the production order:
00129  * SCR=0 -> SMR -> BRR -> settle -> SCR=TE|RE -> SEMR.
00130  * Don't touch SCMR (reset default 0xF2 is correct). */
00131 *sci9_scr() = k_scr_disabled;
00132 *sci9_smr() = k_smr_8n1_async;
00133 *sci9_brr() = k_brr_115200;
00134
00135 /* Wait roughly one bit time before enabling TX/RX. */
00136 for (volatile uint32_t i = 0U; i < (uint32_t)k_brr_settle_loops; i++) {
00137     __asm__ volatile("nop");
00138 }
00139
00140 *sci9_scr() = k_scr_txrx_enabled;
00141 *sci9_semr() = k_semr_default;
00142 }
00143
00144 /*
=====
00145 * Polled write: spin until TDRE then drop byte into TDR.
00146 *
=====
*/
00147 void sci9_debug_putc(char c)
00148 {
00149     while ((*sci9_ssr() & (uint8_t)k_ssr_tdre) == 0U) {
00150         /* spin */
00151     }
00152     *sci9_tdr() = (uint8_t)c;
00153 }
00154
00155 void sci9_debug_puts(const char *s)
00156 {
00157     if (s == (const char *)0) {
00158         return;
00159     }
00160     while (*s != '\0') {
00161         if (*s == '\n') {
00162             sci9_debug_putc('\r');
00163         }
00164         sci9_debug_putc(*s++);
00165     }
00166 }
00167
00168 void sci9_debug_puthex32(uint32_t v)
00169 {
00170     static const char hex_digits[] = "0123456789ABCDEF";
00171     sci9_debug_putc('0');
00172     sci9_debug_putc('x');
00173     for (int8_t shift = (int8_t)k_hex_shift_init32; shift >= 0;
00174          shift -= (int8_t)k_hex_shift_step) {
00175         sci9_debug_putc(hex_digits[(v » (uint8_t)shift) & (uint8_t)k_hex_nibble_mask]);
00176     }
00177 }
00178
00179 void sci9_debug_puthex16(uint16_t v)
00180 {
00181     static const char hex_digits[] = "0123456789ABCDEF";
00182     sci9_debug_putc('0');
00183     sci9_debug_putc('x');
00184     for (int8_t shift = (int8_t)k_hex_shift_init16; shift >= 0;
00185          shift -= (int8_t)k_hex_shift_step) {
00186         sci9_debug_putc(hex_digits[(v » (uint8_t)shift) & (uint8_t)k_hex_nibble_mask]);
00187     }
00188 }

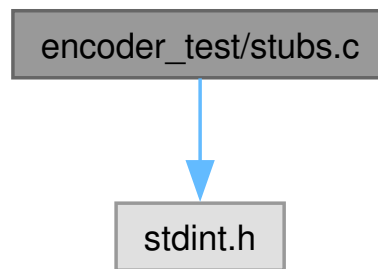
```

4.9 encoder_test/stubs.c File Reference

No-op log/UART stubs for the motor spin test.

```
#include <stdint.h>
```

Include dependency graph for stubs.c:



Functions

- void [rx_log_uart_putc](#) (char c)
- void [rx_log_uart_puts](#) (const char *s)
- void [rx_log_uart_putint](#) (int32_t v)
- void [rx_log_uart_putuint](#) (uint32_t v)
- void [rx_log_uart_puthex](#) (uint32_t v, uint8_t d)
- void [uart_debug_putc](#) (char c)
- void [uart_debug_puts](#) (const char *s)
- void [uart_debug_putint](#) (int32_t v)
- void [uart_debug_putuint](#) (uint32_t v)
- void [uart_debug_puthex](#) (uint32_t v, uint8_t d)

4.9.1 Detailed Description

No-op log/UART stubs for the motor spin test.

The production `rx_motor` / `rx_gptw` / `rx_mpc` libraries call `rx_log_error()` and `RX_ASSERT()` macros that ultimately reduce to a handful of UART log primitives. This bench test does not bring up SCI9 or the framed UART ring buffer, so we satisfy the linker with no-op stubs. `RX_ASSERT` failures still halt (the static-inline `internal_rx_fatal_error` in `rx_check.h` disables interrupts and spins).

SPDX-License-Identifier: MIT

Copyright

Copyright (c) 2026 Locked Inc.

Definition in file [stubs.c](#).

4.9.2 Function Documentation

4.9.2.1 `rx_log_uart_putc()`

```
void rx_log_uart_putc (
    char c)
```

Definition at line 20 of file [stubs.c](#).

```
00020 { (void)c; }
```

4.9.2.2 rx_log_uart_puthex()

```
void rx_log_uart_puthex (  
    uint32_t v,  
    uint8_t d)
```

Definition at line 24 of file [stubs.c](#).

```
00024 { (void)v; (void)d; }
```

4.9.2.3 rx_log_uart_putint()

```
void rx_log_uart_putint (  
    int32_t v)
```

Definition at line 22 of file [stubs.c](#).

```
00022 { (void)v; }
```

4.9.2.4 rx_log_uart_puts()

```
void rx_log_uart_puts (  
    const char * s)
```

Definition at line 21 of file [stubs.c](#).

```
00021 { (void)s; }
```

4.9.2.5 rx_log_uart_putuint()

```
void rx_log_uart_putuint (  
    uint32_t v)
```

Definition at line 23 of file [stubs.c](#).

```
00023 { (void)v; }
```

4.9.2.6 uart_debug_putc()

```
void uart_debug_putc (  
    char c)
```

Definition at line 27 of file [stubs.c](#).

```
00027 { (void)c; }
```

4.9.2.7 uart_debug_puthex()

```
void uart_debug_puthex (  
    uint32_t v,  
    uint8_t d)
```

Definition at line 31 of file [stubs.c](#).

```
00031 { (void)v; (void)d; }
```

4.9.2.8 uart_debug_putint()

```
void uart_debug_putint (
    int32_t v)
```

Definition at line 29 of file [stubs.c](#).

```
00029 { (void)v; }
```

4.9.2.9 uart_debug_puts()

```
void uart_debug_puts (
    const char * s)
```

Definition at line 28 of file [stubs.c](#).

```
00028 { (void)s; }
```

4.9.2.10 uart_debug_putuint()

```
void uart_debug_putuint (
    uint32_t v)
```

Definition at line 30 of file [stubs.c](#).

```
00030 { (void)v; }
```

4.10 stubs.c

[Go to the documentation of this file.](#)

```
00001 /**
00002  * @file stubs.c
00003  * @brief No-op log/UART stubs for the motor spin test.
00004  *
00005  * @details
00006  * The production rx_motor / rx_gptw / rx_mpc libraries call rx_log_error()
00007  * and RX_ASSERT() macros that ultimately reduce to a handful of UART log
00008  * primitives. This bench test does not bring up SCI9 or the framed UART
00009  * ring buffer, so we satisfy the linker with no-op stubs. RX_ASSERT
00010  * failures still halt (the static-inline internal_rx_fatal_error in
00011  * rx_check.h disables interrupts and spins).
00012  *
00013  * SPDX-License-Identifier: MIT
00014  * @copyright Copyright (c) 2026 Locked Inc.
00015  */
00016
00017 #include <stdint.h>
00018
00019 /* rx_log_uart family (selected when RX_IS_SIMULATOR == 0). */
00020 void rx_log_uart_putc(char c) { (void)c; }
00021 void rx_log_uart_puts(const char *s) { (void)s; }
00022 void rx_log_uart_putint(int32_t v) { (void)v; }
00023 void rx_log_uart_putuint(uint32_t v) { (void)v; }
00024 void rx_log_uart_puthex(uint32_t v, uint8_t d) { (void)v; (void)d; }
00025
00026 /* uart_debug family (called unconditionally by internal_rx_fatal_error). */
00027 void uart_debug_putc(char c) { (void)c; }
00028 void uart_debug_puts(const char *s) { (void)s; }
00029 void uart_debug_putint(int32_t v) { (void)v; }
00030 void uart_debug_putuint(uint32_t v) { (void)v; }
00031 void uart_debug_puthex(uint32_t v, uint8_t d) { (void)v; (void)d; }
```